

**IMPLEMENTASI *GRAPH RETRIEVAL*
AUGMENTED GENERATION UNTUK
MENINGKATKAN PRESISI RETRIEVAL DAN
VALIDITAS SINTAKSIS PADA KONFIGURASI
KUBERNETES**

Laporan Tugas Akhir

Oleh

**Jihan Aurelia
18222001**



**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
Maret 2026**

LEMBAR PENGESAHAN

IMPLEMENTASI *GRAPH RETRIEVAL AUGMENTED GENERATION* UNTUK MENINGKATKAN PRESISI RETRIEVAL DAN VALIDITAS SINTAKSIS PADA KONFIGURASI KUBERNETES

Laporan Tugas Akhir

Oleh

Jihan Aurelia
18222001

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Laporan Tugas Akhir ini telah disetujui dan disahkan
di Bandung, pada tanggal 17 Maret 2026

Pembimbing

Dr. Ir. Dimitri Mahayana, M.Eng.

NIP. 196808091991021001

PERNYATAAN ORISINALITAS

Saya yang bertanda tangan di bawah ini menyatakan dengan sesungguhnya bahwa:

1. Tugas Akhir ini adalah hasil karya saya sendiri yang dibuat dengan sebenarnya sesuai dengan kaidah ilmiah dan etika akademik.
2. Semua sumber rujukan dan data yang saya gunakan dalam penyusunan laporan tugas akhir ini, baik yang berupa kutipan langsung maupun tidak langsung, telah saya cantumkan sumbernya dengan baik dan benar sesuai dengan ketentuan penulisan laporan tugas akhir.
3. Isi laporan ini belum pernah diajukan pada program pendidikan di institusi mana pun.

Apabila di kemudian hari terbukti bahwa laporan ini melanggar hal-hal di atas, saya bersedia menerima sanksi sesuai dengan peraturan yang berlaku di Institut Teknologi Bandung.

Bandung, 17 Maret 2026

Jihan Aurelia

NIM: 18222001

PERNYATAAN PENGGUNAAN AI

Saya yang bertanda tangan di bawah ini menyatakan bahwa dalam penulisan dokumen Tugas Akhir ini, saya menggunakan alat bantu kecerdasan artifisial (AI) untuk beberapa aspek tertentu di dalam proses penulisan, seperti yang tercantum dalam tabel berikut.

No.	Jenis	Alat Bantu	Tingkat	Tempat
1	Memeriksa ejaan dan tata bahasa	Grammarly	Tinggi	Semua bab
2	Pembuatan teks	Microsoft Co-pilot	Sedang	Bab II hal. 15
3	Pembuatan grafik, gambar, atau ilustrasi	Tidak ada	Tidak ada	Tidak ada
4	Pencarian informasi atau referensi	...	...	...
5	Penyusunan bahan diskusi	...	...	...
6	Penyusunan kode program	...	...	...
7	Penyusunan formula atau perhitungan matematis	...	...	...
8	Penyusunan konsep desain atau algoritma	...	...	...
9	Penyusunan analisis data	...	...	...
10	Penyusunan kesimpulan atau rekomendasi	...	...	...

Bandung, 17 Maret 2026

Jihan Aurelia
NIM: 18222001

ABSTRAK

Implementasi *Graph Retrieval Augmented Generation* untuk Meningkatkan Presisi Retrieval dan Validitas Sintaksis pada Konfigurasi Kubernetes

oleh

Jihan Aurelia

18222001

Proses manual dalam rantai pasok jeruk di tingkat lapak, yang mencakup sortir, timbang, dan catat, terbukti tidak efisien, memakan waktu, dan rentan terhadap kesalahan. Meskipun solusi berbasis Internet of Things (IoT) dapat mengatasi masalah ini, arsitektur terpusat konvensional berbasis cloud justru menimbulkan masalah baru berupa latensi tinggi yang menghambat alur kerja real-time. Penelitian ini bertujuan untuk merancang dan mengimplementasikan sebuah purwarupa sistem smart scale berbasis IoT dengan pendekatan desentralisasi (edge computing) untuk meminimalkan latensi dan meningkatkan efisiensi operasional. Metode yang digunakan adalah Model Purwarupa, yang diwujudkan dalam bentuk perangkat keras dengan arsitektur prosesor ganda (Raspberry Pi 5 dan ESP32) yang mampu mengintegrasikan fungsi timbang dan analisis kualitas citra secara simultan. Hasil evaluasi menunjukkan bahwa purwarupa dengan arsitektur edge computing yang diimplementasikan mampu memberikan respons 43,5% lebih cepat (latensi rata-rata 18,05 detik) dibandingkan dengan simulasi arsitektur terpusat. Selain itu, sistem ini berhasil meningkatkan efisiensi waktu kerja secara keseluruhan sebesar 58,32% jika dibandingkan dengan metode manual konvensional. Pengujian Penerimaan Pengguna (UAT) yang melibatkan 30 partisipan juga menunjukkan penerimaan yang sangat positif, dengan skor rata-rata untuk kemudahan penggunaan (5,0/5,0), kejelasan antarmuka (4,87/5,0) dan persepsi kinerja (4,95/5,0). Kesimpulannya, implementasi sistem smart scale dengan arsitektur desentralisasi terbukti merupakan solusi yang valid dan efektif untuk menjawab permasalahan latensi dan inefisiensi di lapak. Sistem ini tidak hanya unggul secara teknis dalam hal performa, tetapi juga fungsional dan dapat diterima dengan baik oleh pengguna akhir, serta berpotensi besar untuk modernisasi rantai pasok agribisnis.

Kata kunci: Smart Scale, Internet of Things, Edge Computing, Rantai Pasok Jeruk.

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, saya dapat menyelesaikan penulisan Laporan Tugas Akhir yang berjudul "Implementasi *Graph Retrieval Augmented Generation* untuk Meningkatkan Presisi Retrieval dan Validitas Sintaksis pada Konfigurasi Kubernetes". Laporan ini disusun sebagai salah satu syarat untuk menyelesaikan program Sarjana di Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Bandung, 20 Mei 2026

Penulis

Jihan Aurelia

DAFTAR ISI

LEMBAR PENGESAHAN	iii
PERNYATAAN ORISINALITAS	v
PERNYATAAN PENGGUNAAN AI	vii
ABSTRAK	ix
KATA PENGANTAR	xi
DAFTAR ISI	xiii
DAFTAR GAMBAR	xv
DAFTAR TABEL	xvii
DAFTAR PERSAMAAN	xix
DAFTAR ALGORITMA	xxi
DAFTAR <i>LISTING</i>	xxiii
DAFTAR SIMBOL	xxv
DAFTAR SINGKATAN	xxix
I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	3
I.3 Tujuan	3
I.4 Batasan Masalah	4
I.5 Metodologi	5
II STUDI LITERATUR	9
II.1 Arsitektur <i>Cloud</i> Modern dan Tantangan Kompleksitas	9
II.2 Kubernetes: Platform Orkestrasi Kontainer untuk Aplikasi <i>Cloud-Native</i>	10
II.2.1 Gambaran Umum Arsitektur	10
II.2.2 <i>Resource Model</i> dan Hierarki	12
II.2.3 Manifes YAML dan <i>Infrastructure as Code</i>	14
II.3 <i>Large Language Models</i> (LLM)	14
II.3.1 Definisi dan Kapabilitas	14
II.3.2 <i>Prompt Engineering</i>	15
II.4 <i>Retrieval-Augmented Generation</i> (RAG)	15
II.4.1 Arsitektur <i>Retrieval-Augmented Generation</i>	16
II.4.2 Paradigma RAG	18
II.4.3 Relevansi RAG terhadap Dokumentasi Kubernetes API	19
II.5 <i>Knowledge Graph</i> dan GraphRAG	19
II.5.1 Definisi dan Struktur	19
II.5.2 Integrasi KG dalam RAG pada Tahap Perancangan Solusi	22
II.5.3 GraphRAG: Integrasi <i>Knowledge Graph</i> dan <i>Large Language Models</i>	23
II.6 Evaluasi Sistem Model	24
II.6.1 Evaluasi Retrieval	24
II.6.2 Evaluasi Generasi Jawaban	25
II.7 Penelitian Terkait	26

II.7.1	<i>Empowering LLMs by hybrid retrieval-augmented generation for domain-centric Q&A in smart manufacturing</i>	26
II.7.2	<i>Using Graphs to Improve the Interpretability of API Documentation for BIM Authoring Software</i>	27
II.7.3	<i>KnowledGPT: Enhancing Large Language Models with Retrieval and Storage Access on Knowledge Bases</i>	28
III	ANALISIS MASALAH	31
III.1	Analisis Kondisi Saat Ini	31
III.2	Analisis Kebutuhan	31
III.2.1	Identifikasi Masalah Pengguna	32
III.2.2	Kebutuhan Fungsional	32
III.2.3	Kebutuhan Nonfungsional	32
III.3	Analisis Pemilihan Solusi	32
III.3.1	Alternatif Solusi	32
III.3.2	Analisis Penentuan Solusi	33
IV	PERANCANGAN	35
V	IMPLEMENTASI	37
VI	EVALUASI	39
VI.1	Metode Evaluasi	39
VI.2	Hasil Evaluasi	39
VI.3	Pembahasan Hasil Evaluasi	39
VI.4	Evaluasi Kesalahan Penulisan yang Sering Terjadi	39
VI.4.1	Penggunaan Kata "di mana" atau "dimana"	39
VI.4.2	Penggunaan Kata "sedangkan" dan "sehingga"	39
VI.4.3	Penggunaan Istilah yang Tidak Baku	40
VI.4.4	Pemisah Desimal dan Ribuan	40
VI.4.5	Daftar atau List	40
VI.4.6	Penggunaan Kata "masing-masing" dan "setiap"	41
VII	PENUTUP	43
VII.1	Kesimpulan	43
VII.2	Saran	43
Lampiran A	KODE PROGRAM	49
A.1	Perangkat Lunak untuk Akuisisi Data dari Sensor Ultrasonik	49
A.2	Perangkat Lunak untuk Pengolahan Data Akuisisi dan Visualisasi Hasil Pengukuran Jarak	50
Lampiran B	HASIL SURVEI LAPANGAN	51
B.1	Foto Survei Lokasi 1	51
B.2	Foto Survei Lokasi 2	51
B.3	Transkrip Wawancara dengan Narasumber	51

DAFTAR GAMBAR

I.1	Tahapan model referensi CRISP-DM (Niakšu 2015)	5
II.1	Arsitektur klaster Kubernetes yang terdiri atas <i>Control Plane</i> dan <i>Worker Nodes</i> , dihubungkan melalui <i>kube-apiserver</i> (The Kubernetes Authors 2024b)	11
II.2	Arsitektur <i>Knowledge Graph</i> (Yu dkk. 2021)	20
II.3	Model arsitektur integrasi KG dalam RAG (Knollmeyer, Caymazer, dan Grossmann 2025)	29

DAFTAR TABEL

VI.1 Contoh penggunaan kata ”sedangkan” dan ”sehingga”	40
--	----

DAFTAR PERSAMAAN

DAFTAR ALGORITMA

DAFTAR *LISTING*

A.1	Binary search code	49
-----	------------------------------	----

DAFTAR SIMBOL

Notasi	Deskripsi	Pemakaian pertama kali
α	Koefisien bobot untuk <i>loss function</i> , tingkat signifikansi statistik	75
β	Koefisien bobot untuk <i>Binary Cross-Entropy loss</i>	75
λ	Koefisien bobot untuk CTC <i>loss</i> , parameter regularisasi	75
θ	Parameter model <i>neural network</i>	75
σ	Deviasi standar	75
μ	Rata-rata (<i>mean</i>)	75
ϵ	<i>Token blank</i> /kosong dalam CTC, konstanta kecil untuk stabilitas numerik	75
Δ	Delta (selisih/perubahan nilai)	75
π	<i>Alignment path</i> dalam CTC	75
π_t	<i>Token</i> pada <i>timestep</i> t dalam <i>alignment path</i>	76
∇ atau ∂	Operator gradien atau turunan parsial	75
$\sum$	Operator penjumlahan	75
$\prod$	Operator perkalian	75
$\int$	Operator integral	75
argmax	Argumen yang memaksimalkan fungsi	76
$\sim$	Terdistribusi menurut	75
$\rightarrow$	Menuju atau konvergen ke	75
$\mathcal{L}$	Fungsi <i>loss</i> (kerugian)	75
$\mathcal{L}_{total}$	Total <i>loss function</i>	75
$\mathcal{L}_{adversarial}$ atau $\mathcal{L}_{adv}$	<i>Adversarial loss</i>	75
$\mathcal{L}_{reconstruction}$	<i>Reconstruction loss</i>	75
$\mathcal{L}_{L1}$	L1 <i>loss</i> (<i>Mean Absolute Error</i>)	75
$\mathcal{L}_{L2}$	L2 <i>loss</i> (<i>Mean Squared Error</i>)	75
$\mathcal{L}_{CTC}$	CTC (<i>Connectionist Temporal Classification</i>) <i>loss</i>	75
$\mathcal{L}_{perceptual}$	<i>Perceptual loss</i>	75

$\mathcal{L}_{pixel}$	<i>Pixel-wise loss</i>	75
$\mathcal{L}_{BCE}$	<i>Binary Cross-Entropy loss</i>	75
$\mathcal{L}_{GAN}$	<i>GAN loss</i>	75
$\mathcal{L}_{cycle}$	<i>Cycle consistency loss</i>	75
G	<i>Generator dalam GAN</i>	75
D	<i>Discriminator dalam GAN</i>	75
$G(z)$	<i>Output dari generator dengan input z</i>	75
$D(x)$	<i>Output dari discriminator dengan input x</i>	75
$G_{A \rightarrow B}$	<i>Generator dari domain A ke B</i>	75
$G_{B \rightarrow A}$	<i>Generator dari domain B ke A</i>	75
$V(D, G)$	<i>Fungsi nilai (value function) dalam GAN</i>	75
x	<i>Sampel data asli, input citra</i>	75
y	<i>Label atau ground truth, kondisi dalam conditional GAN</i>	75
z	<i>Noise vector atau representasi laten</i>	
$\mathbb{E}$	<i>Ekspektasi atau nilai harapan</i>	75
p_{data}	<i>Distribusi data asli</i>	75
p_z	<i>Distribusi prior (noise)</i>	75
$p(y x)$	<i>Probabilitas bersyarat output y given input x</i>	75
$p_t(\pi_t x)$	<i>Probabilitas token pada timestep t</i>	
$\mathcal{B}$	<i>Fungsi penciutan (collapse function) dalam CTC</i>	75
$\mathcal{B}^{-1}$	<i>Fungsi invers penciutan dalam CTC</i>	75
T	<i>Jumlah timestep atau panjang sequence</i>	
$ x - G(y) _1$	<i>L1 distance antara ground truth dan generated image</i>	75
$ x - G(y) _2$	<i>L2 distance antara ground truth dan generated image</i>	75
$N \times N$	<i>Ukuran patch dalam PatchGAN</i>	
d	<i>Cohen's d (effect size)</i>	75
n	<i>Jumlah sampel</i>	75
p	<i>p-value (nilai probabilitas statistik)</i>	75

r	Koefisien korelasi	75
R^2	Koefisien determinasi	
$O(n)$	Notasi Big-O untuk kompleksitas linear	75
$O(n^2)$	Notasi Big-O untuk kompleksitas kuadratik	75

DAFTAR SINGKATAN

Singkatan	Deskripsi	Pemakaian pertama kali
AI	<i>Artificial Intelligence</i>	1
CNN	<i>Convolutional Neural Network</i>	2
GPU	<i>Graphics Processing Unit</i>	14

BAB I

PENDAHULUAN

I.1 Latar Belakang

Transformasi industri teknologi informasi menuju arsitektur *cloud-native* kini menjadi kebutuhan fundamental, bukan sekadar tren. Gartner (2021) memproyeksikan bahwa pada tahun 2025, lebih dari 95% beban kerja digital akan di-*deploy* pada platform *cloud-native*. Angka ini meningkat tajam dari hanya 30% pada tahun 2021. Fondasi utama dari arsitektur *cloud-native* ini adalah penggunaan teknologi kontainer yang memungkinkan aplikasi dikemas dan dijalankan secara efisien. Namun, seiring dengan peningkatan skala sistem, pengelolaan ribuan kontainer secara manual tidak lagi memungkinkan sehingga dibutuhkan sebuah alat otomatisasi. Untuk menjembatani kebutuhan kompleks inilah, Kubernetes (K8s) hadir di pusat ekosistem dan mengukuhkan posisinya sebagai standar industri *de facto* untuk orkestrasi kontainer.

Berdasarkan laporan survei *Cloud Native Computing Foundation* (CNCF) tahun 2023, Kubernetes mendominasi pangsa pasar global karena ekosistemnya yang matang, dukungan komunitas yang luas, serta fleksibilitas dalam mengelola infrastruktur berskala besar secara deklaratif (Cloud Native Computing Foundation 2023). Pada tahun 2022, sekitar 58% pengguna telah menjalankan Kubernetes di lingkungan produksi dan 23% lainnya masih dalam tahap evaluasi (total 81%). Pada tahun 2023, angka tersebut meningkat menjadi 66% pengguna di produksi dengan 18% dalam evaluasi (total 84%). Peningkatan signifikan ini menunjukkan bahwa Kubernetes semakin menjadi fondasi utama dalam pengelolaan layanan *cloud-native* modern.

Meskipun Kubernetes mendominasi lanskap orkestrasi kontainer, platform ini menghadirkan tantangan operasional yang signifikan berupa kompleksitas konfigurasi berbasis berkas YAML. Paradigma deklaratif Kubernetes menuntut pengembang untuk menulis ratusan baris kode yang verbos dan hierarkis demi mendefinisikan *desired state* dari sebuah aplikasi yang mencakup berbagai spesifikasi parameter dari Kubernetes. Kompleksitas ini diperparah oleh interdependensi antar-objek yang

sangat ketat namun sering kali tidak terlihat secara eksplisit (*implicit dependencies*) yang mengakibatkan kesalahan kecil seperti ketidakcocokan *label selector* dapat memicu kegagalan sistem berantai. Akibat beban kognitif yang tinggi ini, kerumitan YAML tidak hanya menjadi hambatan skalabilitas utama bagi 65% penggunanya, tetapi juga memicu epidemi miskonfigurasi operasional dan keamanan; tercatat sekitar 80% insiden operasional berakar pada kompleksitas ini, dan 18% berkas konfigurasi sumber terbuka terbukti mengandung pelanggaran standar keamanan kritis industri (Rahman dkk. 2023).

Situasi tersebut berdampak langsung pada pemanfaatan *Large Language Models* (LLM) untuk menghasilkan kode maupun konfigurasi teknis Kubernetes. Walaupun model seperti GPT-4 menawarkan kemudahan dalam memahami dokumentasi API, kemampuan tersebut tidak selalu sejalan dengan ketepatan semantik. Studi evaluatif oleh Liu dkk. (2024) menunjukkan bahwa LLM dapat menghasilkan kode yang secara sintaksis tampak benar, tetapi keliru secara semantis. Dalam lingkungan *production-grade*, kesalahan semantik semacam ini tidak hanya bersifat minor, tetapi berpotensi memicu hilangnya layanan, gangguan orkestrasi, hingga *downtime* berskala besar.

Upaya mitigasi telah dilakukan melalui *Retrieval-Augmented Generation* (RAG) berbasis vektor untuk menambahkan konteks dokumentasi saat LLM menjawab pertanyaan teknis. Namun, penelitian oleh Wan dkk. (2025) mengungkapkan bahwa pendekatan vektor RAG hanya mengutamakan kesamaan *embedding* sehingga gagal menangkap relasi kausal, hierarkis, dan berbasis aturan teknis yang diperlukan untuk memahami konfigurasi domain Kubernetes. Pendekatan vektor RAG hanya mencapai 63,4% *exact match accuracy* dan 69,8% *context precision*, menunjukkan rendahnya ketepatan semantik dalam jawaban sistem. *Embedding* yang dilatih dari korpus umum juga mengabaikan terminologi teknis spesifik sehingga menghasilkan jawaban yang tampak relevan secara teks, tetapi tidak tepat secara domain.

Sebagai solusi, Wan dkk. (2025) mengusulkan penggabungan representasi pengetahuan terstruktur melalui *Knowledge Graph* (KG) untuk memberikan fondasi penalaran relasional yang eksplisit. Integrasi KG dalam sistem RAG menghasilkan peningkatan kinerja yang signifikan, mencapai 77,8% *exact match accuracy* dan 76,5% *context precision*, yaitu peningkatan lebih dari 14,4 poin persentase dibanding pendekatan vektor RAG. KG tidak hanya meningkatkan presisi pencarian informasi, tetapi juga memungkinkan *semantic alignment* melalui pembatasan ontologis sehingga model tidak hanya memilih teks yang mirip, tetapi juga mengakses

pengetahuan yang benar dalam domain.

Dengan demikian, pendekatan Graph-RAG menawarkan fondasi penalaran teknis yang lebih dapat dipertanggungjawabkan dan mampu menurunkan risiko halusinasi semantik berbasis domain, khususnya pada sistem *cloud-native* yang sensitif terhadap kesalahan konfigurasi. Berdasarkan urgensi tersebut, penelitian ini bertujuan untuk membangun sistem *Retrieval-Augmented Generation* (RAG) berbasis *Knowledge Graph* yang dikhususkan untuk dokumentasi Kubernetes. Pendekatan ini diharapkan mampu meningkatkan akurasi *retrieval* terhadap pertanyaan yang membutuhkan penalaran struktural dengan mengekspresikan relasi ontologis antar objek konfigurasi Kubernetes. Selain menghasilkan jawaban teknis berupa penjelasan dan konfigurasi YAML yang valid, sistem juga dirancang untuk memberikan jejak penalaran graf sebagai mekanisme pendeteksi halusinasi. Hal ini memungkinkan setiap keluaran tidak hanya relevan secara tekstual, tetapi juga dapat diverifikasi kesesuaiannya terhadap skema objek Kubernetes.

I.2 Rumusan Masalah

Penelitian ini bertujuan untuk menjawab pertanyaan-pertanyaan berikut,

1. Bagaimana membangun *knowledge graph* dari spesifikasi Kubernetes guna merepresentasikan struktur dan relasi antar-objek konfigurasi secara komprehensif?
2. Bagaimana merancang mekanisme GraphRAG yang memanfaatkan representasi struktural dalam *knowledge graph* untuk meningkatkan presisi *retrieval* pada konfigurasi Kubernetes?
3. Bagaimana perbandingan kinerja antara GraphRAG, *Vanilla* RAG, dan *Vector-based* RAG dalam meningkatkan presisi *retrieval* serta validitas sintaksis berkas konfigurasi Kubernetes?

I.3 Tujuan

Berdasarkan rumusan masalah yang telah dijelaskan pada Bab I.2, tujuan dalam penulisan penelitian ini adalah sebagai berikut,

1. Membangun *knowledge graph* dari spesifikasi OpenAPI Kubernetes (swagger.json) melalui *pipeline* ekstraksi guna memetakan entitas beserta relasi hierarkis dan referensial antar-objek konfigurasi.
2. Mengembangkan mekanisme GraphRAG dengan menelusuri jalur relasional (*graph traversal*) berdasarkan kueri pengguna, kemudian menginjeksikan *reasoning path* tersebut ke dalam *Large Language Model* (LLM) untuk menghasilkan jawaban konsisten sesuai logika domain Kubernetes.

3. Mengevaluasi kinerja metode GraphRAG menggunakan parameter *context metrics*, *faithfulness*, dan *syntactic validity*, serta membandingkannya dengan *Vanilla* RAG maupun *Vector-based* RAG dalam meningkatkan presisi *retrieval* dan validitas sintaksis keluaran YAML menggunakan model generatif GPT-4o-mini.

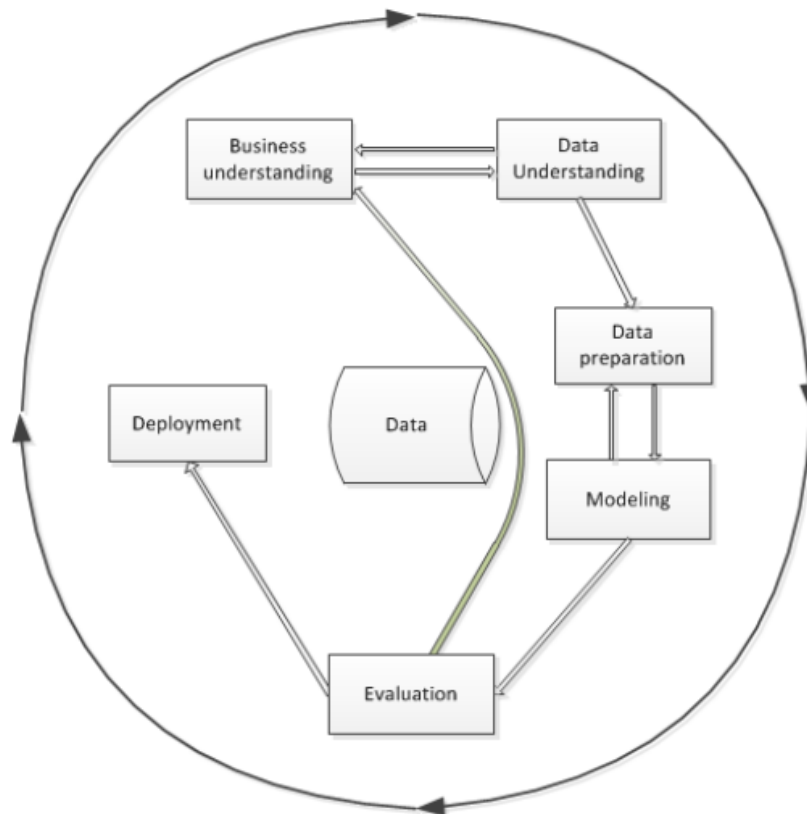
I.4 Batasan Masalah

Agar penelitian ini tetap fokus dan *feasible* dalam lingkup Tugas Akhir, batasan-batasan berikut akan diterapkan,

1. Data penelitian bersumber secara eksklusif dari spesifikasi resmi Kubernetes OpenAPI (swagger.json) versi v1.30 tanpa melakukan *web scraping* dokumentasi HTML. Ekstraksi data dibatasi murni pada blok *definitions* dengan mengeliminasi blok operasional protokol API (seperti *paths*, *parameters*, dan *responses*) guna mencegah *context noise* pada LLM.
2. *Knowledge graph* yang dibangun hanya merepresentasikan struktur skema (*schema structure*) dan tidak mencakup perilaku operasional (*runtime behavior*) maupun logika validasi dengan fokus utama penelitian untuk menguji validitas struktur sintaksis YAML.
3. Luaran penelitian berupa manifes Kubernetes YAML murni (*plain YAML*) berdasarkan *OpenAPI Specification*, dilengkapi jejak penalaran graf (*graph reasoning path*) guna mendeteksi halusinasi LLM.
4. Luaran file YAML tidak diuji langsung pada klaster Kubernetes nyata (*actual cluster*). Pengujian sistem murni berfokus pada validitas sintaksis menggunakan pustaka *pyyaml* serta evaluasi kinerja berdasarkan dataset uji tervalidasi pakar.
5. Model LLM yang digunakan berasal dari *pre-trained models* yang diakses melalui API (GPT-4o-mini) tanpa proses *fine-tuning*.
6. Algoritma *graph traversal* dibatasi maksimal 4 lompatan (*hops*) sesuai analisis topologi objek Kubernetes. Pembatasan ini bertujuan mengoptimalkan *Context Precision* serta efisiensi beban komputasi pada sumber daya lokal.
7. Proses pengolahan dan pra-pemrosesan data dibatasi pada lingkungan **Google Colaboratory** dan **Jupyter Notebook**.
8. Kinerja komputasi dan hasil eksperimen didasarkan pada perangkat keras spesifik (PC dengan CPU AMD Ryzen 5, GPU AMD Radeon RX 6750XT, RAM 16 GB) dan mungkin bervariasi pada konfigurasi lain.

I.5 Metodologi

Pelaksanaan Tugas Akhir ini mengadopsi kerangka kerja *Cross-Industry Standard Process for Data Mining* (CRISP-DM) yang dapat dilihat pada Gambar I.1 sebagai metodologi utama. CRISP-DM dipilih karena memberikan proses terstruktur, iteratif, dan dapat diadaptasi untuk pengembangan sistem berbasis analisis data dan representasi pengetahuan (Niakšu 2015). Tahapan metodologi yang digunakan terdiri dari enam fase berikut,



Gambar I.1 Tahapan model referensi CRISP-DM (Niakšu 2015)

1. Pemahaman Bisnis (*Business Understanding*)

Tahap ini bertujuan untuk mengidentifikasi kebutuhan serta masalah inti yang ingin diselesaikan melalui pemanfaatan data. Dalam konteks penelitian ini, permasalahan yang diangkat adalah ketidaktepatan jawaban teknis yang dihasilkan *Large Language Models* (LLM) ketika merujuk dokumentasi Kubernetes, terutama pada konfigurasi YAML yang bersifat struktural dan rentan mengandung halusinasi. Oleh karena itu, kebutuhan penelitian diarahkan pada peningkatan akurasi penalaran struktural pada proses *retrieval* serta mitigasi halusinasi konfigurasi pada sistem *cloud-native*.

2. Pemahaman Data (*Data Understanding*)

Fase ini berfokus pada pengumpulan dan eksplorasi sumber data untuk mema-

hami struktur, kualitas, serta batasannya. Pada penelitian ini, data diperoleh dari spesifikasi resmi Kubernetes OpenAPI yang dianalisis untuk memahami jenis entitas konfigurasi, atribut yang dimiliki, serta relasi hierarkis dan referensi antar objek. Pemahaman ini menentukan objek Kubernetes yang akan dimodelkan dalam *Knowledge Graph*.

3. **Persiapan Data (*Data Preparation*)**

Tahapan ini mencakup pembersihan dan transformasi data agar siap digunakan dalam proses pemodelan. Dalam penelitian ini, dilakukan ekstraksi otomatis dari spesifikasi OpenAPI Kubernetes (JSON/YAML) untuk membentuk representasi entitas, yaitu relasi yang dapat dimodelkan sebagai *Knowledge Graph*. Proses ini mencakup identifikasi atribut, pemetaan hierarki *parent-child*, serta penghubungan *cross-resource references*. Hasil akhirnya berupa struktur graf yang terstandarisasi dan siap digunakan pada mekanisme *retrieval*.

4. **Pemodelan (*Modeling*)**

Fase ini bertujuan untuk menerapkan algoritma yang sesuai guna menyelesaikan masalah yang telah dirumuskan. Pada penelitian ini, dibangun mekanisme *graph-guided retrieval* yang menelusuri graf menggunakan *graph traversal* berdasarkan kueri pengguna dan menghasilkan *reasoning path*. Jalur penalaran tersebut kemudian diinjeksikan sebagai konteks pada proses RAG untuk memandu LLM dalam menghasilkan jawaban serta konfigurasi YAML yang sesuai dengan logika sistem Kubernetes.

5. **Evaluasi (*Evaluation*)**

Tahap evaluasi bertujuan menilai kualitas model dan membandingkannya dengan pendekatan alternatif menggunakan metrik terukur. Dalam penelitian ini, kinerja sistem dievaluasi menggunakan metrik *context precision*, *context recall*, *faithfulness*, dan *Syntactic Validity*, kemudian dibandingkan dengan dua *baseline*, yaitu *Vanilla LLM* dan *Vector-based RAG*. Evaluasi dilakukan untuk membuktikan peningkatan relevansi konteks dan penurunan halusinasi semantik pada jawaban teknis.

6. **Penyebaran (*Deployment*)**

Fase ini berkaitan dengan penyajian hasil implementasi dalam bentuk prototipe dan dokumentasi. Pada penelitian ini, penyebaran diwujudkan dalam bentuk antarmuka interaktif sederhana berupa CLI yang menampilkan jawaban, konfigurasi YAML, serta *reasoning path* sebagai bukti penalaran graf. Selain itu, dokumentasi lengkap disusun sebagai bentuk penyebaran akademik yang memuat rancangan sistem, hasil evaluasi, serta rekomendasi penelitian

selanjutnya.

BAB II

STUDI LITERATUR

II.1 Arsitektur *Cloud* Modern dan Tantangan Kompleksitas

Pergeseran paradigma dari arsitektur monolitik menuju layanan mikro (*microservices*) kini telah menjadi fondasi utama dalam pengembangan aplikasi *cloud-native* modern. Berbeda dengan pendekatan arsitektur tradisional, gaya arsitektur layanan mikro mendistribusikan logika bisnis secara masif ke dalam puluhan hingga ratusan layanan kecil independen. Distribusi fungsionalitas ini memunculkan kebutuhan akan mekanisme penerapan (*deployment*) mandiri. Sebagai solusi, layanan mikro secara alami terintegrasi dengan platform berbasis kontainer karena setiap layanan dapat dikemas dan didistribusikan di dalam kontainernya masing-masing (Soldani, Tamburri, dan Van Den Heuvel 2018). Walaupun memberikan keuntungan operasional yang signifikan berupa portabilitas lintas *cloud*, skalabilitas tingkat tinggi, dan kebebasan pemilihan teknologi bagi pengembang, ekosistem kontainer ini memunculkan paradoks kompleksitas baru (Soldani, Tamburri, dan Van Den Heuvel 2018).

Dalam lingkungan terdistribusi yang melibatkan banyak kontainer ini, orkestrasi penerapan layanan mikro menjadi sebuah tantangan tersendiri. Setiap layanan mikro di dalam kontainer berkomunikasi melalui mekanisme antarmuka pemrograman aplikasi (API) seperti protokol HTTP. Spesifikasi API beserta konfigurasi orkestrasinya bertindak sebagai kontrak komunikasi esensial antar-layanan (Soldani, Tamburri, dan Van Den Heuvel 2018). Kontrak struktural ini sama sekali tidak boleh dilanggar guna menjaga stabilitas operasional serta memastikan layanan-layanan tersebut dapat terus saling berinteraksi secara dinamis (Soldani, Tamburri, dan Van Den Heuvel 2018). Ketergantungan yang sangat ketat di bawah kendali sistem orkestrasi ini menuntut pemahaman komprehensif terhadap interaksi antar-layanan sebagai kunci utama keberhasilan operasional aplikasi *cloud*.

Akibatnya, dokumentasi API dan manifes konfigurasi orkestrasi mengalami lonjakan kompleksitas. Sebagaimana diidentifikasi oleh Soldani, Tamburri, dan Van Den Heuvel (2018), pemahaman kognitif yang keliru terhadap arsitektur dan interaksi

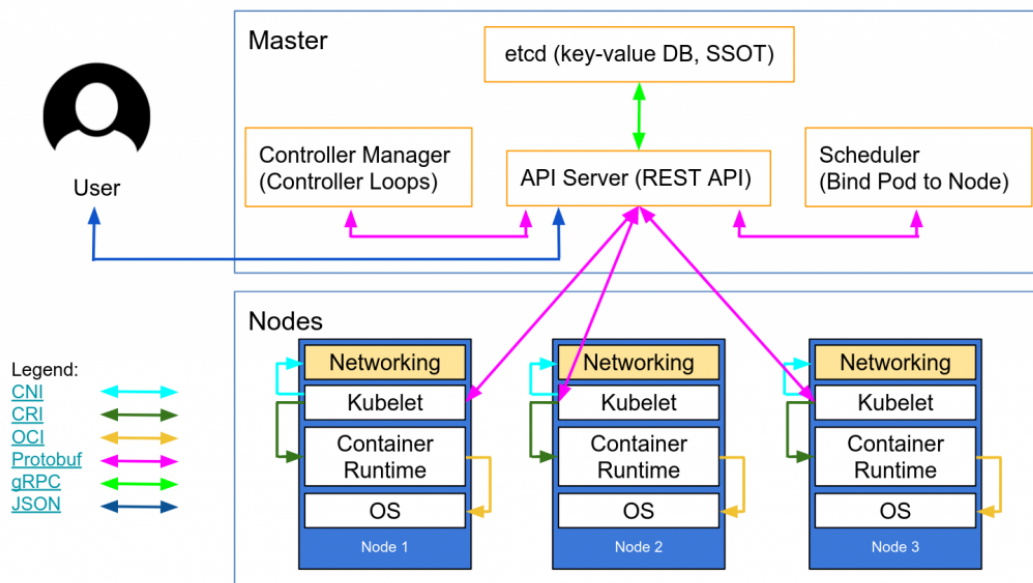
kontrak API ini sering kali berujung pada kegagalan operasional. Jika tidak diisolasi dengan tepat dalam sistem orkestrasi, kegagalan komunikasi pada satu layanan mikro dapat memicu rentetan kegagalan pada layanan lain yang bergantung padanya. Kondisi ini pada akhirnya menghasilkan fenomena kegagalan sistem berantai (*cascading failures*) yang melumpuhkan seluruh aplikasi (Soldani, Tamburri, dan Van Den Heuvel 2018).

II.2 Kubernetes: Platform Orkestrasi Kontainer untuk Aplikasi *Cloud-Native*

Kubernetes merupakan platform orkestrasi kontainer yang dirancang untuk mengotomatisasi proses *deployment*, *scaling*, dan manajemen aplikasi berbasis kontainer dalam lingkungan *cloud-native* (The Kubernetes Authors 2024b). Platform ini menyediakan mekanisme deklaratif berbasis konfigurasi yang memungkinkan pengguna mendefinisikan keadaan sistem yang diinginkan (*desired state*) melalui berkas konfigurasi, sementara Kubernetes menjaga agar keadaan tersebut tetap terpenuhi melalui rangkaian komponen pengendali (*controller loops*).

II.2.1 Gambaran Umum Arsitektur

Arsitektur Kubernetes dibangun atas konsep klaster yang terdiri dari dua kelompok komponen utama, yaitu *Control Plane* dan *Worker Nodes*. *Control Plane* berfungsi sebagai pengendali utama yang membuat keputusan bisnis aplikasi, misalnya penjadwalan, pemantauan, serta konsistensi status. *Worker Nodes* merupakan entitas komputasi fisik/virtual yang menjalankan kontainer sebagai satuan aplikasi. Kedua komponen ini berkomunikasi secara internal melalui API server yang menjadi pintu utama seluruh permintaan sistem.



Gambar II.1 Arsitektur kluster Kubernetes yang terdiri atas *Control Plane* dan *Worker Nodes*, dihubungkan melalui *kube-apiserver* (The Kubernetes Authors 2024b)

1. *Control Plane*

Control Plane bertanggung jawab mengelola keadaan (*state*) kluster secara keseluruhan. Komponen utamanya meliputi,

- kube-apiserver** sebagai gerbang akses seluruh operasi manipulasi objek konfigurasi melalui API,
- etcd** sebagai basis data *key-value* yang menyimpan informasi konfigurasi dan status,
- kube-scheduler** yang menentukan Node penempatan Pod berdasarkan ketersediaan sumber daya,
- kube-controller-manager** yang berisi sekumpulan kontroler untuk menjaga kesesuaian antara kondisi aktual dan *desired state*. Melalui mekanisme deklaratif ini, Kubernetes secara otomatis memperbaiki status objek yang tidak sesuai dan memastikan setiap aplikasi berjalan sesuai aturan konfigurasi yang ditentukan.

2. *Worker Nodes*

Worker Nodes merupakan lapisan eksekusi tempat aplikasi berjalan dalam bentuk Pod. Node terdiri atas tiga komponen inti:

- kubelet** yang bertugas mengeksekusi Pod sesuai *specification* yang diterima dari *Control Plane*,
- kube-proxy** yang menyediakan layanan *network forwarding* dan *load balancing* sederhana antar Pod dalam kluster,

- c. **container runtime** yang bertanggung jawab menjalankan kontainer. Dengan demikian, Node menjalankan komputasi terdistribusi yang dikontrol oleh *Control Plane* melalui komunikasi berbasis API.

II.2.2 Resource Model dan Hierarki

Dalam *resource model* Kubernetes, terdapat beberapa objek inti pembangun aplikasi yang paling sering digunakan:

- **Pod**: Unit komputasi terkecil sebagai pembungkus kontainer aplikasi
- **Deployment**: Pengatur replika dan proses pembaruan versi untuk mencapai ketersediaan tinggi
- **Service**: Penyedia titik akses jaringan yang stabil untuk aplikasi di dalam *Pod*
- **Ingress**: Pengatur rute lalu lintas eksternal masuk ke dalam kluster
- **ConfigMap**: Penyimpan variabel lingkungan dan konfigurasi non-sensitif
- **Secret**: Pengelola data sensitif seperti kredensial dan token
- **PersistentVolumeClaim**: Permintaan penyimpanan persisten untuk data yang harus bertahan
- **ServiceAccount**: Identitas yang digunakan oleh *Pod* untuk berinteraksi dengan API server
- **Role** dan **RoleBinding**: Kontrol akses berbasis peran (RBAC)
- **HorizontalPodAutoscaler**: Pengatur skala otomatis berdasarkan metrik penggunaan

Antar-objek ini saling terikat melalui berbagai jenis relasi yang dapat dikategorikan sebagai berikut:

1. Relasi Struktural (*Structural Relationships*)

- **HAS_PROPERTY**: Hubungan antara objek induk dengan properti skema yang dimilikinya (misal: *Deployment* memiliki properti *spec*)
- **EXTENDS**: Hubungan pewarisan skema dari mekanisme *allOf* (misal: *Deployment* extends *ObjectMeta*)
- **ONE_OF**: Hubungan tipe union yang mengharuskan pemilihan satu dari beberapa opsi (misal: *Volume* memilih satu sumber penyimpanan)
- **ANY_OF**: Hubungan tipe alternatif yang memungkinkan satu atau lebih opsi (misal: *EnvFromSource* dapat menggunakan *ConfigMap* atau *Secret*)

2. Relasi Workload (*Workload Relationships*)

- **CONTAINS_POD_TEMPLATE**: Hubungan antara controller (*Deployment*, *ReplicaSet*, *DaemonSet*, *StatefulSet*, *Job*) dengan template *Pod* yang dikelolanya

- CONTAINS_JOB_TEMPLATE: Hubungan antara *CronJob* dengan template *Job* yang dijadwalkan
- HAS_CONTAINER: Hubungan antara *PodSpec* dengan definisi *Container* yang dijalankan di dalamnya

3. Relasi Penyimpanan (*Storage Relationships*)

- CLAIMS_VOLUME: Hubungan antara *StatefulSet* dengan *PersistentVolumeClaim* untuk penyimpanan persisten
- MOUNTS_VOLUME: Hubungan antara *PodSpec* dengan *Volume* yang di-mount ke kontainer
- USES_STORAGE_CLASS: Hubungan antara *PersistentVolumeClaim* dengan *StorageClass* yang menentukan jenis penyimpanan

4. Relasi Konfigurasi (*Configuration Relationships*)

- LOADS_CONFIGMAP: Hubungan antara *Container* dengan *ConfigMap* yang digunakan untuk injeksi konfigurasi
- USES_SECRET: Hubungan antara *PodSpec* dengan *Secret* untuk data sensitif

5. Relasi Jaringan (*Networking Relationships*)

- SELECTS_POD: Hubungan antara *Service* dengan *Pod* yang dipilih melalui label selector
- ROUTES_TO_SERVICE: Hubungan antara *Ingress* dengan *Service* tujuan rute lalu lintas

6. Relasi RBAC (*Role-Based Access Control Relationships*)

- BINDS_ROLE: Hubungan antara *RoleBinding/ClusterRoleBinding* dengan *Role/ClusterRole* yang diikat
- BINDS_SERVICE_ACCOUNT: Hubungan antara *RoleBinding/ClusterRoleBinding* dengan *ServiceAccount* yang diberi izin
- USES_SERVICE_ACCOUNT: Hubungan antara *PodSpec* dengan *ServiceAccount* yang digunakan untuk identitas

7. Relasi Autoscaling (*Autoscaling Relationships*)

- SCALES_RESOURCE: Hubungan antara *HorizontalPodAutoscaler* dengan workload target (*Deployment, StatefulSet, ReplicaSet*) yang diskalakan

Pemetaan relasi kepemilikan (*ownership*) dan ketergantungan fungsional (*dependencies*) inilah yang akan secara langsung dikonversi menjadi skema graf pengetahuan (*knowledge graph*) pada penelitian ini. Sebagai contoh, sebuah *Deployment* secara hierarkis memiliki sub-sumber daya *PodSpec* melalui relasi CONTAINS_JOB_TEMPLATE, dan *Pod* tersebut memiliki ketergantungan operasional terhadap *ConfigMap* tertentu melalui relasi LOADS_CONFIGMAP. Total terdapat 21 jenis edge yang terdistribusi

dalam 7 kategori relasi, membentuk ontologi lengkap untuk generasi YAML Kubernetes.

II.2.3 Manifes YAML dan *Infrastructure as Code*

Pengembang berinteraksi dengan model sumber daya Kubernetes melalui pendekatan *Infrastructure as Code* (IaC) (The Kubernetes Authors 2024a). Pendekatan ini mewajibkan pengguna untuk mendeklarasikan wujud infrastruktur yang diinginkan (*desired state*) ke dalam bentuk dokumen teks berformat YAML. Setiap manifes YAML standar wajib memiliki empat atribut struktural utama agar dapat diterima oleh API Kubernetes:

1. `apiVersion`: Menentukan versi skema API pembungkus objek.
2. `kind`: Mendefinisikan jenis entitas sumber daya.
3. `metadata`: Memuat data identifikasi unik seperti nama, label pengelompokan, dan *namespace*.
4. `spec`: Berisi spesifikasi teknis rinci penentu sifat dan bentuk hierarki objek tersebut.

Penyusunan manifes YAML menuntut pemahaman terkait batasan skema. Pengembangan harus bisa membedakan secara presisi antara ruas wajib (*required*) penentu validitas dan ruas opsional (*optional*) penambah fitur khusus. Praktik terbaik (*best practices*) dalam penulisan IaC menyarankan modularitas dokumen, konsistensi penggunaan label, dan pemisahan logika aplikasi dari data konfigurasi. Sebuah struktur manifes YAML yang mematuhi kelengkapan ruas wajib (*field requirements*) dan valid secara sintaksis inilah yang menjadi representasi target luaran yang akan dibangkitkan oleh sistem generasi pada penelitian ini.

II.3 *Large Language Models* (LLM)

II.3.1 Definisi dan Kapabilitas

Large Language Model (LLM) merupakan model kecerdasan buatan berbasis jaringan saraf berskala besar yang dilatih pada korpus teks *multi-domain* untuk mempelajari representasi bahasa alami secara mendalam (Wan dkk. 2025). Melalui proses pelatihan tersebut, LLM mampu menginterpretasikan konteks linguistik yang kompleks serta menghasilkan keluaran berupa teks yang koheren, kontekstual, dan adaptif terhadap berbagai jenis tugas seperti *question answering*, penalaran, serta peringkasan teks.

Meskipun unggul dalam pemahaman bahasa alami, LLM memiliki keterbatasan faktual yang signifikan ketika diterapkan pada domain teknis, terutama terhadap pengetahuan yang presisi, bersifat prosedural, serta memerlukan landasan ontologis.

Beberapa keterbatasan yang disorot oleh Wan dkk. (2025) meliputi,

- a. *Hallucination*, yaitu kemampuan menghasilkan jawaban yang secara linguistik meyakinkan tetapi tidak memiliki landasan fakta yang jelas sehingga berisiko menyesatkan proses pengambilan keputusan teknis.
- b. Keterbatasan cakupan pengetahuan, LLM hanya mengandalkan data pelatihan awal sehingga tidak selalu mencakup informasi domain yang mutakhir.
- c. Ketidadaan atribusi sumber, yaitu LLM tidak dapat memberikan justifikasi atau referensi eksplisit terhadap jawaban yang dihasilkan.

Keterbatasan tersebut menjadi alasan utama perlunya integrasi antara LLM dan arsitektur *retrieval-augmented generation* (RAG).

II.3.2 *Prompt Engineering*

Prompt engineering merupakan teknik perancangan instruksi masukan bagi LLM untuk mengendalikan keluaran model berdasarkan tujuan tugas, cakupan konteks, serta batasan informasi yang diizinkan (Wan dkk. 2025). Dalam domain teknis, teknik ini tidak hanya berperan sebagai instruksi linguistik, tetapi juga sebagai representasi pengetahuan eksplisit yang mendefinisikan bagaimana LLM harus memanfaatkan sumber informasi domain.

Efektivitas keluaran LLM dipengaruhi secara signifikan oleh strategi *prompt engineering* berikut,

- a. Spesifikasi tugas yang eksplisit, termasuk definisi peran model dan batasan prosedural, misalnya instruksi untuk membatasi jawaban pada proses manufaktur tertentu.
- b. Penyediaan konteks terstruktur, melalui penyisipan entitas domain, hubungan semantik, serta cuplikan dokumen terkait yang relevan dengan pertanyaan.
- c. Pemberian batasan (*constraint injection*) yang secara eksplisit membatasi ruang solusi berdasarkan pengetahuan terbatas dalam dokumen yang valid.
- d. Penyusunan format jawaban, misalnya dalam bentuk tabel, daftar berhierarki, atau penjelasan berbasis pembuktian prosedural.

Strategi tersebut tidak hanya mengontrol keluaran, tetapi juga mencegah LLM menafsirkan konteks di luar pengetahuan domain sehingga berperan sebagai mekanisme mitigasi terhadap fenomena *hallucination*.

II.4 *Retrieval-Augmented Generation* (RAG)

Retrieval-Augmented Generation (RAG) merupakan arsitektur pemrosesan bahasa alami yang mengombinasikan mekanisme *Information Retrieval* (IR) dengan kemampuan generatif *Large Language Models* (LLM). Pendekatan ini dikembangkan

untuk mengatasi keterbatasan memori parametris pada model bahasa, khususnya dalam menangani tugas yang bersifat *knowledge-intensive*, yaitu model rentan menghasilkan informasi yang keliru atau tidak berbasis fakta (*hallucination*) (Antonio Moreno-Cediel 2025).

Berbeda dengan metode generatif konvensional yang hanya mengandalkan pengetahuan tersimpan selama proses pelatihan, RAG memanfaatkan basis pengetahuan non-parametris melalui proses pencarian (*retrieval*) sehingga jawaban yang dihasilkan lebih akurat dan relevan terhadap konteks masukan.

II.4.1 **Arsitektur *Retrieval-Augmented Generation***

Arsitektur *Retrieval-Augmented Generation* (RAG) merupakan pendekatan hibrida yang menggabungkan kemampuan penalaran *Large Language Models* (LLM) dengan basis pengetahuan eksternal melalui proses pencarian semantik. Struktur ini dirancang untuk mengurangi *hallucination*, meningkatkan akurasi fakta, serta mendukung pembaruan pengetahuan secara dinamis (Gao dkk. 2024). Secara umum, alur kerja RAG terdiri atas empat tahapan teknis utama yang berurutan dan saling bergantung, yaitu *Data Ingestion & Indexing*, *Retrieval*, *Augmentation*, dan *Generation*.

1. ***Data Ingestion & Indexing***

Tahapan ini bersifat *offline* karena dilakukan sebelum sistem menjawab pertanyaan. Tujuan utamanya adalah mentransformasi data mentah menjadi basis pengetahuan terstruktur yang dapat dengan mudah dicari oleh mesin. Prosesnya terdiri dari langkah-langkah berikut,

a. ***Akuisisi Data***

Sistem mengumpulkan data dari berbagai sumber seperti dokumentasi teknis, *API specification*, artikel ilmiah, *knowledge base*, atau korpus web.

b. ***Preprocessing***

Pembersihan dokumen dilakukan melalui normalisasi format, penghapusan *noise*, serta ekstraksi metadata sehingga struktur informasi menjadi homogen dan konsisten.

c. ***Chunking***

Dokumen panjang dipecah menjadi unit informasi kecil (*chunks*) berukuran 200–500 token agar sesuai dengan batas konteks LLM dan dapat di-*retrieve* secara efisien.

d. ***Embedding***

Setiap *chunk* diubah menjadi representasi numerik berbasis vektor meng-

gunakan *sentence embedding models* seperti MiniLM, BERT, atau struktur densitas modern lainnya.

e. *Indexing*

Hasil *embedding* disimpan pada *vector store* (misalnya FAISS, Chroma, Weaviate, Pinecone) sehingga pencarian dapat dilakukan berdasarkan kedekatan semantik antar vektor.

2. *Retrieval*

Proses *retrieval* dilakukan ketika pengguna mengajukan pertanyaan. Sistem akan mencari konteks paling relevan melalui mekanisme berikut,

a. *Embedding* Pertanyaan

Pertanyaan pengguna diubah menjadi vektor menggunakan model *embedding* yang sama dengan proses indexing.

b. *Similarity Search*

Dilakukan pencocokan vektor menggunakan metrik jarak seperti *cosine similarity*, *dot-product*, atau *Euclidean distance*.

c. *Top-k Selection*

Sistem memilih sejumlah dokumen teratas (umumnya $k = 3$ atau $k = 5$) yang memiliki skor kemiripan tertinggi dan mengandung konteks yang berpotensi menjawab pertanyaan.

Kualitas *retrieval* yang baik menjadi faktor kunci dalam menurunkan risiko *hallucination* pada LLM karena jawaban dibatasi oleh fakta yang relevan.

3. *Augmentation*

Tahap *augmentation* bertugas menyisipkan konteks hasil *retrieval* ke dalam *prompt* secara sistematis sehingga dapat dipahami oleh LLM. Prosesnya mencakup tahapan berikut,

a. *Concatenation*

Sistem menggabungkan pertanyaan pengguna dengan konten dokumen yang ditemukan.

b. *Formatting*

Struktur informasi dapat diberikan dalam format teks baku, tabel, *JSON schema*, *instruction-style*, atau bentuk terstruktur lainnya.

c. *Constraint Injection*

Sistem memberikan batasan eksplisit, misalnya “jawab hanya menggunakan konteks yang diberikan, jangan menambah informasi baru”, untuk mencegah LLM berhalusinasi.

II.4.2 Paradigma RAG

Retrieval-Augmented Generation berkembang dalam tiga paradigma utama, yaitu *Naive RAG*, *Advanced RAG*, dan *Modular RAG*. Ketiga paradigma ini mencerminkan evolusi dari *pipeline* sederhana menuju arsitektur yang dapat diadaptasi untuk berbagai kebutuhan, mulai dari peningkatan akurasi sampai fleksibilitas sistem (Gao dkk. 2024).

1. *Naive RAG*

Paradigma ini merupakan bentuk paling dasar dari RAG. *Pipeline* umumnya terdiri dari tiga langkah, yaitu *indexing*, *retrieval*, dan *generation*. Dokumen melalui proses *chunking*, kemudian direpresentasikan menggunakan *embedding* dan disimpan pada *vector store*. Ketika pertanyaan diajukan, sistem mengambil k dokumen paling relevan menggunakan pencarian berbasis kesamaan semantik, lalu model generatif menyusun jawaban berdasarkan konteks tersebut. Keterbatasan utama pendekatan ini adalah ketergantungan penuh terhadap kualitas *chunk* dan hasil *retrieval* yang dapat mengarah pada jawaban yang tidak akurat.

2. *Advanced RAG*

Paradigma ini melakukan optimasi pada dua fase utama, yaitu *pre-retrieval* dan *post-retrieval*. Tahap *pre-retrieval* meliputi peningkatan kualitas indeks seperti pengayaan metadata, peningkatan granularitas *chunk*, serta teknik *query rewriting* dan *query expansion* untuk meningkatkan relevansi hasil pencarian. Sementara itu, *post-retrieval* dilakukan melalui *reranking* serta *context compression* agar konteks yang terkirim ke model lebih berfokus pada informasi penting saja. Paradigma ini masih mengikuti alur linear tetapi menghasilkan akurasi yang lebih tinggi dibanding *Naive RAG*.

3. *Modular RAG*

Paradigma ini memperluas arsitektur RAG menjadi sistem yang dapat dikustomisasi melalui penambahan atau penggantian modul. Contoh dari modul yang ada yaitu *Search Module* (untuk menggabungkan pencarian dari *search engine*, database, maupun *knowledge graph*), *Memory Module* (yang memanfaatkan memori internal LLM), *Routing Module* (yang memilih jalur eksekusi berdasarkan tipe pertanyaan), serta *Task Adapter Module* (yang menyesuaikan proses dengan kebutuhan tugas tertentu). Modular RAG juga memungkinkan pola *iterative retrieval* dan *adaptive retrieval* sehingga menjadikannya paradigma paling fleksibel untuk aplikasi domain spesifik.

II.4.3 Relevansi RAG terhadap Dokumentasi Kubernetes API

Dokumentasi Web API, termasuk Kubernetes, memiliki karakteristik unik berupa kombinasi deskripsi sintaks terstruktur (misalnya skema JSON/YAML) dan penjelasan semantik dalam bentuk teks alami. Penelitian Kotstein dan Decker (2024) menunjukkan bahwa pemrosesan semantik terhadap dokumentasi API dapat dilakukan tanpa ontologi formal dengan memetakan makna berdasarkan nama parameter, struktur hierarkis, dan deskripsi bahasa alami.

Dalam konteks Kubernetes, komponen API seperti Pod, Service, dan Deployment direpresentasikan melalui skema bertingkat yang bersifat kompleks dan memiliki ketergantungan semantik. Oleh karena itu, implementasi RAG pada domain ini membutuhkan,

1. ***Schema-aware semantic chunking***

Teknik ini memastikan bahwa pemotongan dokumen (*chunking*) dilakukan mengikuti struktur skema objek API. Setiap potongan teks harus mewakili satu entitas atau blok atribut yang utuh sehingga relasi antar-*field* tidak terpisah atau kehilangan konteks. Dengan demikian, representasi vektor yang dihasilkan tetap mempertahankan hierarki objek.

2. ***Path-based serialization***

Teknik ini menyimpan setiap elemen API beserta jalur lengkapnya dalam struktur objek sehingga posisi semantiknya tetap terpelihara.

3. ***Hybrid index retrieval***

Pendekatan ini mengombinasikan pencarian berbasis kesamaan semantik teks dengan pembobotan struktur sintaksis dari objek API. Selain memanfaatkan *embedding* untuk menemukan kemiripan makna antar-teks, sistem juga memberikan prioritas pada elemen yang memiliki signifikansi struktural (misalnya *field* wajib atau relasi antar-objek). Hasilnya, mekanisme *retrieval* menghasilkan konteks yang tidak hanya relevan secara linguistik, tetapi juga benar secara hierarki dan fungsi API.

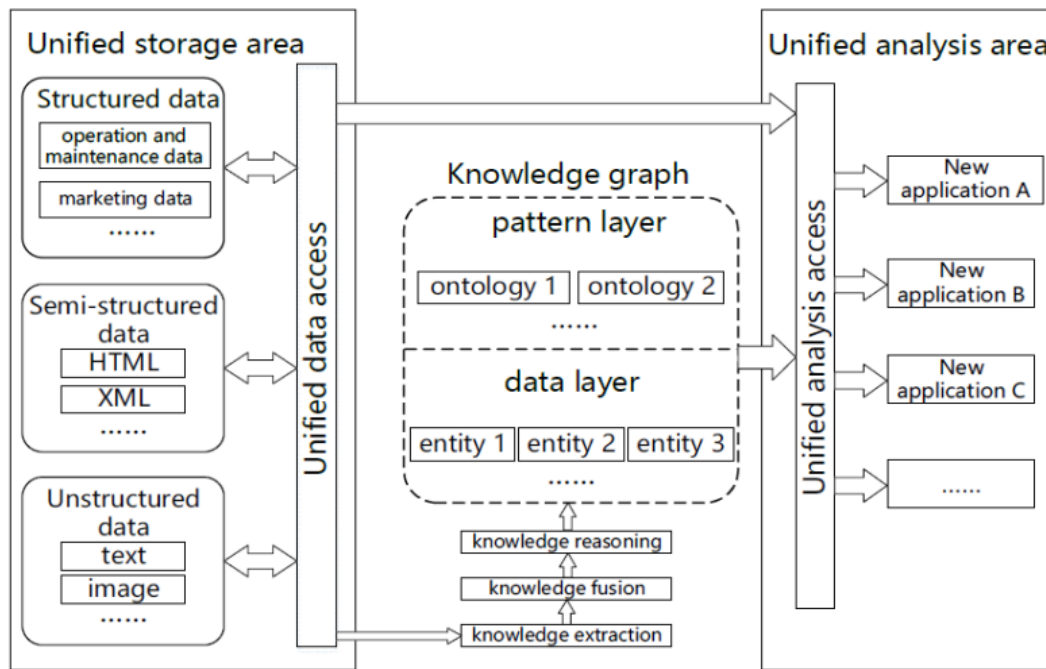
Dengan demikian, penerapan RAG pada dokumentasi Kubernetes tidak hanya berfungsi sebagai mesin pencarian semantik, tetapi juga menjadi pendekatan mitigasi *hallucination*.

II.5 Knowledge Graph dan GraphRAG

II.5.1 Definisi dan Struktur

Knowledge Graph (KG) merupakan representasi pengetahuan dalam bentuk graf yang tersusun dari entitas (*node*) dan relasi (*edge*) yang memiliki makna semantik

(Hofer dkk. 2024). Berbeda dengan basis data konvensional yang mengandalkan skema statis, KG bersifat *schema-flexible* sehingga mampu mengintegrasikan data heterogen dari sumber terstruktur, semi-terstruktur, maupun tidak terstruktur seperti yang ditunjukkan pada Gambar II.2. Struktur KG umumnya dinyatakan melalui *triples* berbentuk (subjek–predikat–objek).



Gambar II.2 Arsitektur *Knowledge Graph* (Yu dkk. 2021)

Menurut Yu dkk. (2021), terdapat dua pendekatan utama dalam pembangunan KG, yaitu *top-down* dan *bottom-up* yang dibedakan berdasarkan urutan perancangan model dan pengisian fakta.

1. *Top-down Construction*

Pada pendekatan ini, proses pembangunan KG dimulai dari definisi model semantik terlebih dahulu. Langkah pertama mencakup perancangan *ontology library* yang memuat kelas entitas, relasi, hierarki, dan aturan semantik. Setelah struktur konseptual ditetapkan, barulah fakta atau data dimasukkan ke dalam basis pengetahuan. Dengan demikian, pendekatan *top-down* mengikuti urutan *model layer* kemudian *data layer*. Pendekatan ini umumnya digunakan pada domain yang membutuhkan konsistensi semantik tinggi seperti medis, keuangan, atau layanan publik.

2. *Bottom-up Construction*

Berbeda dari pendekatan sebelumnya, *bottom-up* mengawali proses konstruksi melalui ekstraksi data langsung dari korpus teks atau sumber data lain. De-

ngan menganalisis frekuensi entitas, dan pola semantik kemudian dibentuk secara bertahap. Artinya, *data layer* dibangun terlebih dahulu, kemudian dilanjutkan dengan penyusunan *pattern layer*. Pendekatan ini didorong oleh ketersediaan data dan sesuai untuk domain yang cepat berubah atau belum memiliki referensi ontologi.

Menurut Hofer dkk. (2024), konstruksi *Knowledge Graph* (KG) modern umumnya dilakukan secara semi-otomatis dan bersifat inkremental. Proses pembangunan KG mencakup sejumlah tugas inti yang dapat dijalankan secara paralel, asinkron, maupun selektif sesuai jenis sumber data dan kebutuhan penggunaan. Tugas-tugas utama tersebut adalah sebagai berikut,

1. **Manajemen Metadata**

Pengelolaan metadata terkait, seperti *provenance* entitas, informasi temporal, struktur data, kualitas data, hingga log proses. Tugas ini bersifat menyeluruh karena diperlukan pada setiap tahapan konstruksi KG.

2. ***Data Acquisition dan Preprocessing***

Pemilihan sumber data yang relevan, akuisisi data, transformasi format, serta pembersihan awal. Tingkat detail proses ini bergantung pada keragaman sumber yang digunakan.

3. **Manajemen Ontologi**

Pembuatan dan pengembangan ontologi secara bertahap untuk mendefinisikan kelas, properti, dan relasi yang membentuk struktur semantik KG.

4. ***Knowledge Extraction (KE)***

Ekstraksi informasi terstruktur dari data tidak terstruktur atau semi-terstruktur menggunakan teknik seperti *Named Entity Recognition*, *entity linking*, dan *relation extraction*.

5. ***Entity Resolution (ER) dan Fusion***

Penyatuan entitas yang merepresentasikan objek yang sama untuk menghindari duplikasi.

6. ***Knowledge Completion***

Pelengkapan KG dengan pengetahuan baru melalui inferensi, prediksi relasi hilang, atau penambahan tipe entitas untuk meningkatkan kelengkapan semantik.

7. ***Quality Assurance (QA)***

Identifikasi masalah kualitas data pada KG serta penerapan strategi perbaikan yang dapat mencakup *validation checks*, konsistensi semantik, atau pembersihan lanjutan.

Proses ini tidak selalu bersifat linear karena urutan eksekusi bergantung pada jenis

sumber data dan beberapa langkah dapat menghilangkan kebutuhan langkah lain, seperti *entity linking* yang dapat menggantikan *entity resolution*. Selain itu, *metadata management* bersifat *cross-cutting* karena berlangsung pada seluruh tahap konstruksi KG.

II.5.2 Integrasi KG dalam RAG pada Tahap Perancangan Solusi

Sistem menggabungkan *Knowledge Graph* (KG) dengan alur RAG yang bertujuan untuk meningkatkan akurasi pencarian, kemampuan *multi-hop reasoning*, serta menjaga keterkaitan struktur dokumen. Arsitektur keseluruhan mengikuti tiga fase utama, yaitu *Indexing*, *Retrieval*, dan *Generation* (Knollmeyer, Caymazer, dan Grossmann 2025). Model arsitektur ditunjukkan pada Gambar II.3.

1. Konstruksi KG dan Ekstraksi Struktur Dokumen (*Indexing*)

Proses awal dimulai dari ekstraksi teks serta identifikasi elemen struktur dokumen. Proses *chunking* tidak hanya memecah teks secara semantik, tetapi juga mempertahankan hierarki dokumen melalui pemberian URI pada setiap *chunk* sehingga dapat ditelusuri kembali ke lokasi asalnya. Pada tahap ini dibentuk dua lapisan graf berikut,

- (a) *Document KG* (DKG) yang memetakan hubungan hierarkis dokumen (bab–subbab–*chunk*).
- (b) *Information KG* (IKG) yang menghubungkan antar-*chunk* melalui *keywords* untuk meningkatkan relevansi semantik pada dokumen.

Seluruh URI *chunk* juga disimpan pada basis data vektor untuk memungkinkan pencarian hibrid antara *dense embedding* dan hubungan semantik.

2. Pencarian Hibrid Berbasis Graf (*Retrieval*)

Setelah dilakukan *initial retrieval* berbasis *dense embedding*, sistem memperluas pencarian melalui operasi graf yang memanfaatkan struktur dokumen dan hubungan semantik. Tiga strategi yang digunakan adalah:

- (a) *Informed Chapter Search* (ICS), yaitu menambahkan *chunks* lain yang berada dalam bab yang sama dengan hasil pencarian awal.
- (b) *Informed Keyword Search* (IKS), yaitu menambahkan *chunks* yang terhubung melalui *keywords* pada IKG.
- (c) *Uninformed Keyword Search* (UKS), yaitu mengekstraksi *keyword* langsung dari pertanyaan pengguna dan mencari seluruh *chunk* terkait pada KG.

3. Pembentukan Jawaban Berbasis Bukti (*Generation*)

Pada fase akhir, sistem menerapkan *pass condition* untuk membatasi jumlah konteks yang diberikan pada LLM sehingga mencegah kelebihan informasi yang berpotensi menurunkan kualitas jawaban. Setelah konteks yang relevan

dipilih, proses *prompt engineering* dilakukan untuk memastikan model menghasilkan jawaban yang benar-benar berbasis bukti. Dalam hal ini, LLM diarahkan untuk menyusun jawaban dengan mengutip secara langsung bagian teks dari *chunk* yang relevan, menyatakan secara eksplisit apabila informasi yang tersedia tidak mencukupi untuk menjawab pertanyaan, serta tidak menyertakan pengetahuan eksternal yang tidak berasal dari konteks masukan pengguna. Pendekatan tersebut meningkatkan tingkat *faithfulness*, meminimalkan risiko *hallucination*, sekaligus memastikan transparansi jejak sumber yang mendukung reliabilitas hasil.

II.5.3 GraphRAG: Integrasi *Knowledge Graph* dan *Large Language Models*

Large Language Models (LLM) dan *Knowledge Graphs* (KG) merupakan dua paradigma representasi pengetahuan yang saling melengkapi. LLM unggul dalam pemrosesan bahasa alami serta generalisasi lintas tugas tetapi rentan terhadap halusinasi fakta. Sebaliknya, KG menawarkan representasi pengetahuan faktual dalam bentuk struktur graf eksplisit yang kuat untuk penalaran simbolik namun kurang mampu menangani pemahaman bahasa alami yang kompleks (Pan dkk. 2024).

Untuk mengatasi kelemahan LLM tersebut, pendekatan *Retrieval-Augmented Generation* (RAG) konvensional sering digunakan. Namun, RAG tradisional bekerja dengan cara memecah dokumen menjadi potongan teks (*chunks*) independen sehingga sering kali menghilangkan informasi struktural dan relasi esensial antar-entitas (Ma dkk. 2025). Sebagai solusi komprehensif atas keterbatasan tersebut, muncullah konsep *Graph Retrieval-Augmented Generation* (GraphRAG).

Berbeda dengan pencarian semantik biasa berbasis vektor teks, GraphRAG memanfaatkan mekanisme penelusuran graf (*graph traversal*) untuk mengambil konteks terstruktur secara utuh sebelum diberikan ke LLM (Han dkk. 2024). Mekanisme ini secara langsung mengekstraksi pengetahuan relevan dari data graf dengan merangkai subgraf atau jalur relasional berdasarkan kueri pengguna. Proses ini mencakup ekstraksi subgraf, penyaringan jalur (*path-filtering*), dan penyempurnaan jalur (*path-refinement*) guna menyajikan konteks penalaran yang akurat (Ma dkk. 2025).

Secara formal, integrasi generatif ini dimodelkan sebagai fungsi yang bersyarat terhadap pengetahuan graf:

$$p_{\theta}(y \mid x, \mathcal{K}) = \sum_{z \in \mathcal{Z}(x, \mathcal{K})} p_{\theta}(y \mid x, z) p(z \mid x, \mathcal{K}), \quad (\text{II.1})$$

dengan $\mathcal{K} = \{(h, r, t) \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}\}$ merepresentasikan KG sebagai himpunan

triples, dan $\mathcal{Z}(x, \mathcal{K})$ merupakan himpunan subgraf atau fakta terstruktur yang ditarik dari graf berdasarkan kueri x . Dalam skema ini, KG bertindak sebagai ruang pengetahuan non-parametrik penyedia panduan penalaran (*reasoning guidelines*), sedangkan LLM bertindak sebagai generator parametrik (Ma dkk. 2025).

Pendekatan GraphRAG ini menjadi sangat krusial dalam domain berskala kompleks seperti manajemen konfigurasi Kubernetes. Dengan memetakan hierarki objek dan referensi silang ke dalam graf, algoritma *traversal* dapat merangkai jejak penalaran (*reasoning path*) yang kohesif. Injeksi konteks terstruktur ini ke dalam perintah LLM terbukti secara efektif menekan tingkat halusinasi, memastikan akurasi faktual, dan mempertahankan validitas sintaksis pada hasil akhir manifes YAML yang dibangkitkan.

II.6 Evaluasi Sistem Model

Untuk menilai kinerja sistem *Retrieval-Augmented Generation* (RAG), evaluasi tidak hanya dilakukan pada jawaban akhir, tetapi pada dua komponen utama: (i) kinerja *retrieval* berbasis *graph*, dan (ii) kinerja generasi jawaban oleh LLM. Menurut Es dkk. (2023), pengukuran kualitas harus dimulai dari konteks yang diambil, karena kesalahan pada tahap *retrieval* akan langsung memengaruhi jawaban meskipun model generatif bekerja dengan baik.

II.6.1 Evaluasi Retrieval

Pada tahap *retrieval*, sistem diukur berdasarkan kemampuan *graph search* dalam mengambil *nodes* yang relevan dan menekan konteks yang tidak diperlukan. Dua metrik yang digunakan adalah *Context Recall* dan *Context Precision*, masing-masing diadaptasi dari Es dkk. (2023), *RAGAS: Context Precision* (2025), dan *RAGAS: Context Recall* (2025).

Context Recall mengukur kemampuan sistem menemukan seluruh konteks yang relevan. Nilai yang rendah mengindikasikan hilangnya bukti penting sehingga jawaban dapat menjadi tidak lengkap atau bahkan salah. Rumusnya diberikan pada Persamaan (II.2).

$$\text{ContextRecall} = \frac{\sum_{c \in \mathcal{C}_{\text{retrieved}}} \mathbb{I}[c \in \mathcal{C}_{\text{relevant}}]}{|\mathcal{C}_{\text{retrieved}}|} \quad (\text{II.2})$$

Sementara itu, **Context Precision** mengukur sejauh mana konteks yang diambil benar-benar relevan. Semakin rendah presisi, semakin banyak *noise* yang masuk dan meningkatkan risiko *hallucination*. Rumus presisi ditunjukkan pada Persamaan

an (II.3).

$$\text{ContextPrecision} = \frac{\sum_{c \in \mathcal{C}_{\text{retrieved}}} \mathbb{I}[c \in \mathcal{C}_{\text{relevant}}]}{|\mathcal{C}_{\text{retrieved}}|} \quad (\text{II.3})$$

Pada kedua persamaan tersebut, $\mathbb{I}[\cdot]$ merupakan fungsi indikator bernilai 1 jika kondisi terpenuhi, dan 0 jika tidak; $\mathcal{C}_{\text{retrieved}}$ adalah konteks yang diambil oleh sistem, sedangkan $\mathcal{C}_{\text{relevant}}$ adalah konteks yang seharusnya diambil. Kedua metrik saling berkorelasi dan membentuk *trade-off* bahwa semakin banyak konteks diambil, *recall* meningkat namun *precision* dapat menurun.

II.6.2 Evaluasi Generasi Jawaban

Tahap generasi oleh LLM tidak dapat hanya dinilai dari kesesuaian semantik, namun harus menjamin bahwa jawaban benar-benar berasal dari konteks yang diambil. Menurut Es dkk. (2023), metrik yang paling penting untuk mengukur integritas jawaban adalah ***Faithfulness***, yaitu sejauh mana jawaban dapat ditelusuri kembali ke konteks pertanyaan tanpa penambahan informasi eksternal.

RAGAS mengukur *Faithfulness Score* (FS) berdasarkan klaim dalam jawaban yang dapat dibuktikan oleh konteks yang ada, sebagaimana dirumuskan pada Persamaan (II.4).

$$\text{Faithfulness} = \frac{\sum_{s \in \mathcal{S}_{\text{claims}}} \mathbb{I}[s \in \mathcal{S}_{\text{supported}}]}{|\mathcal{S}_{\text{claims}}|} \quad (\text{II.4})$$

Pada konteks domain Kubernetes, keluaran sistem berupa file YAML harus benar secara fakta dan valid secara sintaks. Oleh sebab itu, metrik tambahan ***Syntactic Validity*** digunakan untuk memverifikasi apakah keluaran dapat diproses tanpa error, misalnya melalui `kubectl apply --dry-run`. Metriknya bersifat biner, sebagaimana didefinisikan pada Persamaan (II.5) dan dirata-rata pada Persamaan (II.6).

$$\text{ValidSyntax}(y) = \begin{cases} 1, & \text{jika keluaran dapat diparsing/di-deploy tanpa error,} \\ 0, & \text{jika terjadi error sintaks YAML.} \end{cases} \quad (\text{II.5})$$

$$S_{\text{syntax}} = \frac{1}{n} \sum_{i=1}^n \text{ValidSyntax}(y_i) \quad (\text{II.6})$$

Dengan demikian, kualitas sistem RAG pada domain *cloud-native* dinilai secara komprehensif, yaitu *Graph Retrieval* diukur melalui *Context Recall* dan *Precision*, sedangkan Generasi LLM diukur melalui *Faithfulness* dan *Syntactic Validity*. Kombinasi ini memastikan hasil yang tidak hanya akurat, tetapi juga dapat dieksekusi pada lingkungan Kubernetes.

II.7 Penelitian Terkait

II.7.1 *Empowering LLMs by hybrid retrieval-augmented generation for domain-centric Q&A in smart manufacturing*

Penelitian oleh Wan dkk. (2025) mengembangkan pendekatan *hybrid retrieval* yang menggabungkan *Knowledge Graph* (KG) dengan *dense vector retrieval* untuk meningkatkan akurasi *question answering* pada domain *Smart Manufacturing*, khususnya desain untuk manufaktur aditif (*Design for Additive Manufacturing*, DfAM). Pendekatan ini didasari oleh kebutuhan untuk menangkap relasi semantik berbasis ontologi sekaligus memanfaatkan generalisasi representasi vektor. Metode yang diusulkan terdiri dari empat tahapan, yaitu (1) *knowledge collection and chunking* untuk mengumpulkan korpus dan membentuk *embeddings*; (2) konstruksi KG berbasis metadata melalui ekstraksi entitas, relasi, dan atribut domain; (3) penerapan pembatas semantik melalui aturan ontologi untuk menyelaraskan kueri; serta (4) *hybrid retrieval* yang menggabungkan pencarian node dan relasi pada KG dengan pencarian vektor berbobot, dilanjutkan dengan *prompt engineering* berbasis aturan domain.

Hasil penelitian menunjukkan bahwa model hibrida yang dilengkapi dengan aturan *prompt* mencapai skor *Exact Match* (EM) dan *Context Precision* tertinggi dibandingkan dengan pendekatan *KG-only* maupun *vector-only*. Pendekatan *vector-only* memiliki keunggulan pada waktu eksekusi (3–7 detik), tetapi mengalami penurunan akurasi karena ketergantungan pada *generic embedding*. Sebaliknya, pendekatan *KG-only* lebih baik dalam *reasoning* berbasis relasi, namun bergantung pada kelengkapan graf dan memiliki waktu eksekusi yang relatif lebih lambat. Model hibrida dengan *prompt engineering* menghasilkan akurasi tertinggi dengan rentang waktu 8–16 detik. Dengan demikian, ketika prioritas sistem adalah akurasi dan ketelitian informasi, strategi hibrida menjadi pilihan terbaik, walaupun memerlukan optimasi waktu.

Meskipun efektif, studi tersebut masih memiliki beberapa keterbatasan metodologis yang membuka peluang penelitian lanjutan. Pertama, pembaruan KG dilakukan secara manual sehingga rentan terhadap *knowledge drift* pada domain yang dina-

mis. Kedua, tidak terdapat mekanisme *hallucination detection* yang memastikan setiap keluaran model dapat ditelusuri kembali ke sumber. Ketiga, proses *prompt engineering* masih bersifat manual dan tidak dioptimasi secara otomatis. Keempat, penelitian tidak memasukkan teknik *query rewriting* untuk menangani kueri ambigu yang tidak selaras dengan ontologi domain.

II.7.2 Using Graphs to Improve the Interpretability of API Documentation for BIM Authoring Software

Wrabel (2024) menginvestigasi cara meningkatkan interpretabilitas dokumentasi *Application Programming Interface* (API) pada perangkat lunak *Building Information Modeling* (BIM) dengan mengubah dokumentasi API yang tidak terstruktur menjadi representasi graf berbasis *Knowledge Graph* (KG) dan mengintegrasikannya dengan agen *Retrieval-Augmented Generation* berbasis graf (GraphRAG). Tujuan utama penelitian adalah menyediakan cara penelusuran fungsi API yang lebih akurat dan mendukung generasi kode langsung melalui kueri LLM.

Metode penelitian menggabungkan pendekatan deterministik dan berbasis *embedding* untuk konstruksi graf API. Pertama, dokumentasi API dikonversi menjadi struktur *JavaScript Object Notation* (JSON) dan diparsing menjadi entitas serta relasi fungsi secara deterministik. Kedua, bagian dokumentasi yang tidak terstruktur diekstraksi menggunakan model LLM untuk menghasilkan *embedding* yang memetakan entitas semantik tambahan. Ketiga, contoh implementasi kode pada dokumentasi web disertakan ke dalam graf melalui relasi *[USES]* sehingga graf tidak hanya memuat hierarki fungsi tetapi juga konteks penggunaannya. Representasi data ini diwujudkan dalam bentuk *Labeled Property Graph* (LPG) menggunakan Neo4j.

Selanjutnya, agen GraphRAG dibangun dengan tiga jenis alat kueri, yaitu pencarian semantik berbasis *embedding*, kueri graf *multi-hop* menggunakan Cypher, serta pencarian contoh implementasi kode. Agen menentukan urutan eksekusi alat berdasarkan tipe pertanyaan sehingga dapat menghasilkan jawaban berupa penjelasan API, struktur relasi antar fungsi, atau rekomendasi potongan kode. Evaluasi sistem dilakukan pada 36 pertanyaan teknis mengenai API Vectorworks, memperlihatkan bahwa gabungan graf deterministik dan *embedding* unggul dibanding graf berbasis *embedding* penuh.

Hasil eksperimen membuktikan bahwa *hybrid graph* menghasilkan *code suggestion reliability* dan akurasi jawaban teks yang lebih tinggi dibandingkan KG yang dibangun sepenuhnya menggunakan LLM. Penelitian ini juga menunjukkan bahwa penyertaan relasi konteks penggunaan kode (*[USES]*) meningkatkan relevansi reko-

mendasi fungsi API. Walaupun demikian, penelitian masih memiliki keterbatasan seperti kualitas graf bergantung pada konsistensi dokumentasi sumber, biaya komputasi embedding relatif tinggi, dan tidak terdapat mekanisme untuk mendeteksi atau membatasi *hallucination* pada tahap generasi jawaban.

Studi ini memberikan implikasi bahwa transformasi dokumentasi API ke dalam bentuk graf merupakan pendekatan efektif untuk mendukung *reasoning* teknis dan navigasi fungsi perangkat lunak berbasis kueri NLP. Namun, diperlukan mekanisme tambahan seperti *query rewriting*, *context reranking*, dan *hallucination detection* untuk meningkatkan reliabilitas agen RAG dalam domain teknis yang sensitif terhadap kesalahan implementasi.

II.7.3 *KnowledGPT: Enhancing Large Language Models with Retrieval and Storage Access on Knowledge Bases*

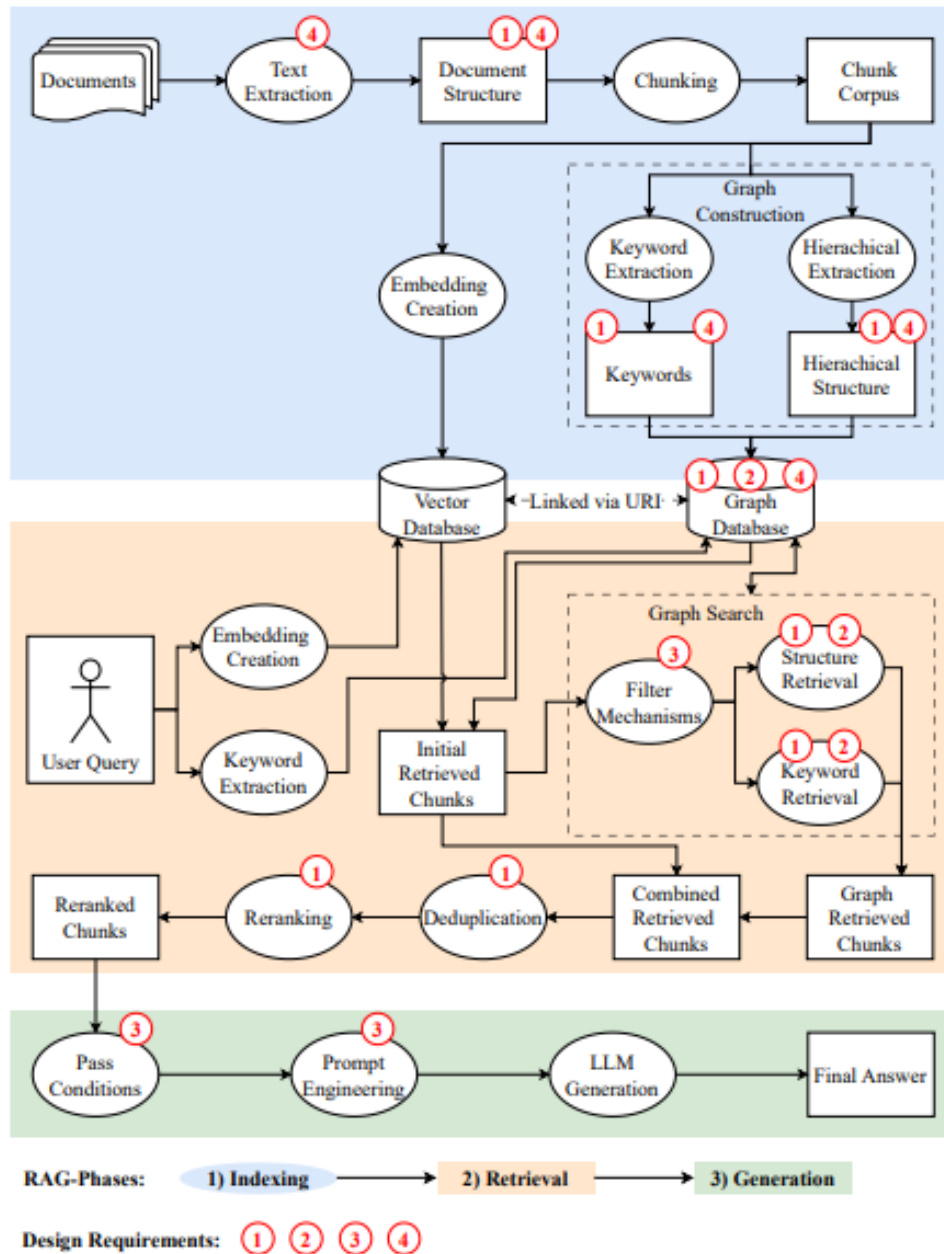
KnowledGPT merupakan pendekatan integrasi LLM dengan *knowledge base* (KB) yang tidak membangun struktur graf eksplisit, tetapi memandu LLM untuk menghasilkan *search code* (*Program-of-Thought*) yang mengeksekusi fungsi akses KB secara deterministik (He dkk. 2024).

Mekanisme dari sistem ini menerjemahkan pertanyaan pengguna q oleh LLM menjadi kode pencarian: `get_entity(e)` atau `find_relation(e_1, e_2)` yang kemudian dijalankan pada KB untuk memperoleh jawaban berbasis fakta. LLM tidak menjawab langsung, tetapi menyimpulkan hanya dari hasil eksekusi fungsi.

KnowledGPT menegaskan tiga prinsip penting yang juga diadopsi dalam desain GraphRAG, yaitu,

1. Penarikan pengetahuan deterministik melalui jalur terdefinisi, bukan probabilistik token prediction.
2. Reduksi halusinasi karena jawaban hanya berasal dari hasil fungsi KB.
3. Kemampuan *multi-hop* untuk memastikan relasi antar entitas dipahami secara struktural.

Prinsip tersebut selaras dengan rancangan penelitian ini, terutama pada *graph topology construction*, relasi $\$ref$, serta *graph traversal-based retrieval* untuk menyediakan konteks teknis yang akurat.



Gambar II.3 Model arsitektur integrasi KG dalam RAG (Knollmeyer, Caymazer, dan Grossmann 2025)

BAB III

ANALISIS MASALAH

Bab ini membahas analisis masalah yang ada pada sistem saat ini, analisis kebutuhan yang diperlukan untuk menyelesaikan masalah tersebut, serta analisis pemilihan solusi yang diusulkan. Secara garis besar, bab ini membahas hal-hal berikut:

1. Deskripsi hasil analisis dari persoalan yang sedang ditangani dalam TA;
2. Penjelasan tentang solusi yang ditawarkan (dapat berupa algoritma, konsep desain, formula, atau gabungannya) pada level konsep dan detail yang cukup untuk merealisasikan solusinya; dan
3. Penjelasan tentang pendekatan yang dipilih dalam menyelesaikan persoalan TA.

Subbab-subbab dalam bab ini dapat disesuaikan dengan kebutuhan topik TA yang dibahas. Berikut adalah contoh pembagian subbab yang dapat digunakan sebagai referensi (judul subbab dapat diubah sesuai dengan hal yang hendak dibahas dalam subbab tersebut).

III.1 Analisis Kondisi Saat Ini

Menurut **laudon2020**<empty citation>, gambarkan terlebih dahulu model konseptual sistem yang ada saat ini. Model konseptual ini berisi berbagai komponen atau subsistem dan interaksi antarsubsistem tersebut. Setelah itu, berikan penjelasan tentang masalah yang ada pada sistem tersebut. Paragraf berikut berisi contoh penjabaran masalah sistem informasi fasilitas kesehatan untuk pasien (**pressman2019**).

III.2 Analisis Kebutuhan

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

III.2.1 Identifikasi Masalah Pengguna

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

III.2.2 Kebutuhan Fungsional

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

III.2.3 Kebutuhan Nonfungsional

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.

III.3 Analisis Pemilihan Solusi

III.3.1 Alternatif Solusi

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Donec odio elit, dictum in, hendrerit sit amet, egestas sed, leo. Praesent feugiat sapien aliquet odio. Integer vitae justo. Aliquam vestibulum fringilla lorem. Sed neque lectus, consectetur at, consectetur sed, eleifend ac, lectus. Nulla facilisi. Pellentesque eget lectus. Proin eu metus. Sed porttitor. In hac habitasse platea dictumst. Suspendisse eu lectus. Ut mi mi, lacinia sit amet, placerat et, mollis vitae,

dui. Sed ante tellus, tristique ut, iaculis eu, malesuada ac, dui. Mauris nibh leo, facilisis non, adipiscing quis, ultrices a, dui.

III.3.2 Analisis Penentuan Solusi

Morbi luctus, wisi viverra faucibus pretium, nibh est placerat odio, nec commodo wisi enim eget quam. Quisque libero justo, consectetur a, feugiat vitae, porttitor eu, libero. Suspendisse sed mauris vitae elit sollicitudin malesuada. Maecenas ultricies eros sit amet ante. Ut venenatis velit. Maecenas sed mi eget dui varius euismod. Phasellus aliquet volutpat odio. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Pellentesque sit amet pede ac sem eleifend consectetur. Nullam elementum, urna vel imperdiet sodales, elit ipsum pharetra ligula, ac pretium ante justo a nulla. Curabitur tristique arcu eu metus. Vestibulum lectus. Proin mauris. Proin eu nunc eu urna hendrerit faucibus. Aliquam auctor, pede consequat laoreet varius, eros tellus scelerisque quam, pellentesque hendrerit ipsum dolor sed augue. Nulla nec lacus.

BAB IV

PERANCANGAN

Ilustrasikan desain konsep solusi dalam bentuk model konseptual dan penjelasan secara ringkas, beserta perbedaannya dengan sistem saat ini. Ilustrasi harus dapat dibandingkan (*before and after*). Karena masih berupa proposal, bab ini hanya berisi gambar desain konsep solusi tersebut dan penjelasan perbandingannya dengan gambar sistem yang ada saat ini (yang tergambar di awal Bab III).

BAB V

IMPLEMENTASI

asdfasdf

BAB VI

EVALUASI

asdfasdf

VI.1 Metode Evaluasi

asdfasdf

VI.2 Hasil Evaluasi

asdfasdf

VI.3 Pembahasan Hasil Evaluasi

VI.4 Evaluasi Kesalahan Penulisan yang Sering Terjadi

VI.4.1 Penggunaan Kata "di mana" atau "dimana"

Banyak yang menuliskan kata "di mana" atau "dimana" sebagai pengganti kata "which" dalam bahasa Inggris. Padahal, penggunaan kata "di mana" atau "dimana" tidak tepat dalam konteks tersebut. Demikian juga untuk kata serupa, misalnya "yang mana". Kata "di mana" atau "dimana" ini harus diganti dengan kata lain, seperti "dengan", "tempat", "yang", dan sebagainya tergantung kalimatnya. Penjelasan lengkap dapat dilihat pada (BPBI).

VI.4.2 Penggunaan Kata "sedangkan" dan "sehingga"

Kata "sedangkan" dan "sehingga" adalah kata hubung atau konjungsi. Konjungsi adalah kata atau ungkapan yang menghubungkan satuan bahasa (kata, frasa, klausa, dan kalimat). Konjungsi dapat dibagi menjadi konjungsi intrakalimat dan antarkalimat. Kata "sedangkan" menghubungkan dua klausa yang bersifat kontrasif, sedangkan "sehingga" menghubungkan dua klausa yang bersifat kausal. Dalam ragam formal, kata hubung "sedangkan" dan "sehingga" hanya dapat digunakan sebagai konjungsi intrakalimat sehingga kedua konjungsi itu **tidak dapat diletakkan pada awal kalimat**. Selain itu, penggunaan kata "sedangkan" harus didahului oleh koma (,), sedangkan kata "sehingga" tidak perlu didahului oleh koma (,). Contoh penggunaan yang benar dan salah dapat dilihat pada Tabel VI.1.

Tabel VI.1 Contoh penggunaan kata "sedangkan" dan "sehingga"

Kata	Salah	Benar
sedangkan	Sedangkan sistem lama masih digunakan oleh banyak pengguna.	Sistem lama masih digunakan oleh banyak pengguna, sedangkan sistem baru belum siap.
sehingga	Sehingga sistem lama masih digunakan oleh banyak pengguna.	Sistem lama masih digunakan oleh banyak pengguna sehingga sistem baru belum siap.

VI.4.3 Penggunaan Istilah yang Tidak Baku

Ada beberapa istilah yang sering digunakan dalam pembicaraan sehari-hari, tetapi tidak baku dalam penulisan ilmiah. Beberapa istilah tersebut antara lain:

1. analisa → analisis
2. eksisting atau existing → yang ada atau saat ini
3. bisnis proses → proses bisnis
4. user → pengguna
5. system → sistem
6. database → basis data
7. aktifitas → aktivitas
8. efektifitas → efektivitas
9. sosial media → media sosial

VI.4.4 Pemisah Desimal dan Ribuan

Tanda pemisah desimal dalam bahasa Indonesia adalah tanda koma, contoh:

1. (Salah) Akurasi naik menjadi 50.6%
2. (Benar) Akurasi naik menjadi 50,6%

VI.4.5 Daftar atau *List*

Ada beberapa aturan penulisan daftar atau *list* yang perlu diperhatikan, antara lain:

- a) Jika memungkinkan, hindari penggunaan "bullet points" atau sejenisnya. Sebaiknya, gunakan angka (1, 2, 3, ...) atau huruf (a, b, c, ...). Dengan demikian, pembaca dapat dengan mudah melihat jumlah *item* atau *list*.
- b) Jika dalam daftar hanya ada satu item, tidak perlu menggunakan nomor urut.
- c) Penjelasan atau deskripsi suatu item sebaiknya menyatu dengan judul item tersebut, tidak berbeda halaman. Contoh yang salah: judul item ada di halaman 10, namun deskripsinya di halaman 11. Sebaiknya pindahkan judul tersebut ke halaman 11.
- d) Jika penjelasan atau deskripsi suatu item cukup panjang, misalnya lebih dari

1 halaman atau terdiri atas beberapa paragraf, sebaiknya setiap item tersebut dijadikan judul subbab, kecuali jika level subbab sudah mencapai level 4.

VI.4.6 Penggunaan Kata "masing-masing" dan "setiap"

Kata "masing-masing" digunakan di belakang kata yang diterangkan, misalnya "Setiap proses menggunakan algoritma masing-masing". Kata "tiap-tiap" atau "setiap" ditempatkan di depan kata yang diterangkan, misalnya "Setiap proses menggunakan algoritma tertentu".

BAB VII

PENUTUP

VII.1 Kesimpulan

Jelaskan secara detail langkah-langkah rencana selanjutnya, hal-hal yang diperlukan atau akan disiapkan, dan risiko dan mitigasinya, yang meliputi:

VII.2 Saran

asdfas

DAFTAR PUSTAKA

- Antonio Moreno-Cediel, David De-Fitero-Dominguez, Eva Garcia-Lopez. Antonio Garcia-Cabot. 2025. "Optimising retrieval performance in RAG systems: A new growing window semantic chunking strategy to address weak semantic boundaries". *Knowledge-Based Systems* 331:114896. <https://doi.org/10.1016/j.knosys.2025.114896>.
- Cloud Native Computing Foundation. 2023. *CNCF Annual Survey 2023*. Diakses pada November 22, 2025. <https://www.cncf.io/reports/cncf-annual-survey-2023/>.
- Es, Shahul, Jithin James, Luis Espinosa-Anke, dan Steven Schockaert. 2023. "RAGAS: Automated Evaluation of Retrieval Augmented Generation". *arXiv preprint arXiv:2309.15217*, <https://arxiv.org/abs/2309.15217>.
- Gao, Yunfan, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, dan Haofen Wang. 2024. "Retrieval-Augmented Generation for Large Language Models: A Survey". Accessed November 25, 2025, *arXiv preprint arXiv:2312.10997*.
- Gartner. 2021. *Gartner Says Cloud Will Be the Centerpiece of New Digital Experiences*. <https://www.gartner.com/en/newsroom/press-releases/2021-11-10-gartner-says-cloud-will-be-the-centerpiece-of-new-digital-experiences>. Press Release, accessed 22 November 2025.
- Han, Haoyu, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A. Rossi, Subhabrata Mukherjee, dan Xianfeng Tang. 2024. "Retrieval-augmented generation with graphs (GraphRAG)". *arXiv preprint arXiv:2501.00309*.
- He, Xiaoxin, Yijun Tian, Yifei Sun, Nitesh Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, dan Bryan Hooi. 2024. *KnowledGPT: Enhancing Large Language Models with Knowledge Base Guided Reasoning*. Technical report. Viewed from user-provided document.

- Hofer, Marvin, Daniel Obraczka, Alieh Saeedi, Hanna Köpcke, dan Erhard Rahm. 2024. “Construction of Knowledge Graphs: Current State and Challenges”. *Information* 15 (8): 509.
- Knollmeyer, Simon, Oğuz Caymazer, dan Daniel Grossmann. 2025. “Document GraphRAG: Knowledge Graph Enhanced Retrieval Augmented Generation for Document Question Answering Within the Manufacturing Domain”. *Electronics* 14 (11): 2102. <https://doi.org/10.3390/electronics14112102>. <https://doi.org/10.3390/electronics14112102>.
- Kotstein, Sebastian, dan Christian Decker. 2024. “RESTBERTa: a Transformer-based question answering approach for semantic search in Web API documentation”. Published online 31 January 2024, *Cluster Computing*, <https://doi.org/10.1007/s10586-023-04237-x>.
- Liu, Zhijie, Anh Nguyen, Junjie Chen, Xin Zhang, Yanjie Li, dan Yingfei Xiong. 2024. “Deep Analysis of LLM-based Code Generation: Correctness, Complexity, and Security”. *IEEE Transactions on Software Engineering*, 1–20. <https://doi.org/10.1109/TSE.2024.3392499>.
- Ma, Chuangtao, Yongrui Chen, Tianxing Wu, Arijit Khan, dan Haofen Wang. 2025. “Large Language Models Meet Knowledge Graphs for Question Answering: Synthesis and Opportunities”. *arXiv preprint arXiv:2505.20099*.
- Niakšu, Olegas. 2015. “CRISP Data Mining Methodology Extension for Medical Domain”. *Baltic Journal of Modern Computing* 3 (2): 92–109. https://www.bjmc.lu.lv/fileadmin/user_upload/lu_portal/projekti/bjmc/Contents/3_2_2_Niaksu.pdf.
- Pan, Shirui, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, dan Xindong Wu. 2024. “Unifying Large Language Models and Knowledge Graphs: A Roadmap”. *IEEE Transactions on Knowledge and Data Engineering* 36 (7): 3580–3599. <https://doi.org/10.1109/TKDE.2024.3352100>.
- RAGAS: Context Precision*. 2025. https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_precision/. Accessed 12 March 2025.
- RAGAS: Context Recall*. 2025. https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_recall/. Accessed 12 March 2025.

- Rahman, A., dkk. 2023. “Empirical Analysis of Security Misconfigurations in Open-Source Kubernetes Manifests”. *Journal of Cloud Computing and Security* 12 (4): 112–128.
- Soldani, Jacopo, Damian Andrew Tamburri, dan Willem-Jan Van Den Heuvel. 2018. “The pains and gains of microservices: A Systematic grey literature review”. *Journal of Systems and Software* 146:215–232. <https://doi.org/10.1016/j.jss.2018.09.082>.
- The Kubernetes Authors. 2024a. *Kubernetes Concepts*. Diakses pada: 2024-05-15. <https://kubernetes.io/docs/concepts/>.
- . 2024b. “Kubernetes Documentation”. Diakses pada February 22, 2025. <https://kubernetes.io/docs/home/>.
- Wan, Yuwei, Zheyuan Chen, Ying Liu, Chong Chen, dan Michael Packianather. 2025. “Empowering LLMs by hybrid retrieval-augmented generation for domain-centric Q&A in smart manufacturing”. *Advanced Engineering Informatics* 65:103212. ISSN: 1474-0346. <https://doi.org/10.1016/j.aei.2025.103212>.
- Wrabel, Tamira. 2024. “Using Graphs to Improve the Interpretability of API Documentation for BIM Authoring Software”. Master’s thesis, Technische Universität München.
- Yu, Jun, Yintie Zhang, Yu Wu, dan Linhui Mao. 2021. “Research on the Practical Application of Visual Knowledge Graph in Technology Service Model and Intelligent Supervision”. *Journal of Physics: Conference Series* 1982:012040.

LAMPIRAN A

KODE PROGRAM

Pada umumnya, kode program tidak perlu disertakan di dalam laporan tugas akhir karena kode program biasanya sangat panjang dan sudah disimpan dalam bentuk berkas-berkas elektronik terpisah di repositori daring. Namun, jika ada bagian kode program singkat yang penting untuk ditulis dalam laporan tugas akhir, bagian tersebut dapat disertakan di dalam laporan.

A.1 Perangkat Lunak untuk Akuisisi Data dari Sensor Ultrasonik

```
1 % -- Example of pseudocode and source code listing --
2 % -- Gunakan minipage agar listing tidak terpotong ke halaman
   berikutnya --
3
4
5
6 \begin{minipage}{\textwidth}
7 \begin{lstlisting}[caption={Iterative binary search in Python},
   label={lst:binary_iter}, frame=single, captionpos=t, language=
   Python]
8 def binary_search(arr, target):
9     """
10     Performs binary search on a sorted array.
11
12     Args:
13         arr: A sorted list of elements
14         target: The element to search for
15
16     Returns:
17         Index of target if found, -1 otherwise
18     """
19     left = 0
20     right = len(arr) - 1
21
22     while left <= right:
23         mid = (left + right) // 2
24
```

```

25     if arr[mid] == target:
26         return mid
27     elif arr[mid] < target:
28         left = mid + 1
29     else:
30         right = mid - 1
31
32     return -1
33
34
35 \end{lstlisting}
36 \end{minipage}

```

Listing A.1 Binary search code

A.2 Perangkat Lunak untuk Pengolahan Data Akuisisi dan Visualisasi Hasil Pengukuran Jarak

asdjfkjashdkjfas

LAMPIRAN B

HASIL SURVEI LAPANGAN

B.1 Foto Survei Lokasi 1

Teks survei lokasi 1.

B.2 Foto Survei Lokasi 2

Teks survei lokasi 2.

B.3 Transkrip Wawancara dengan Narasumber

Teks transkrip wawancara.

